

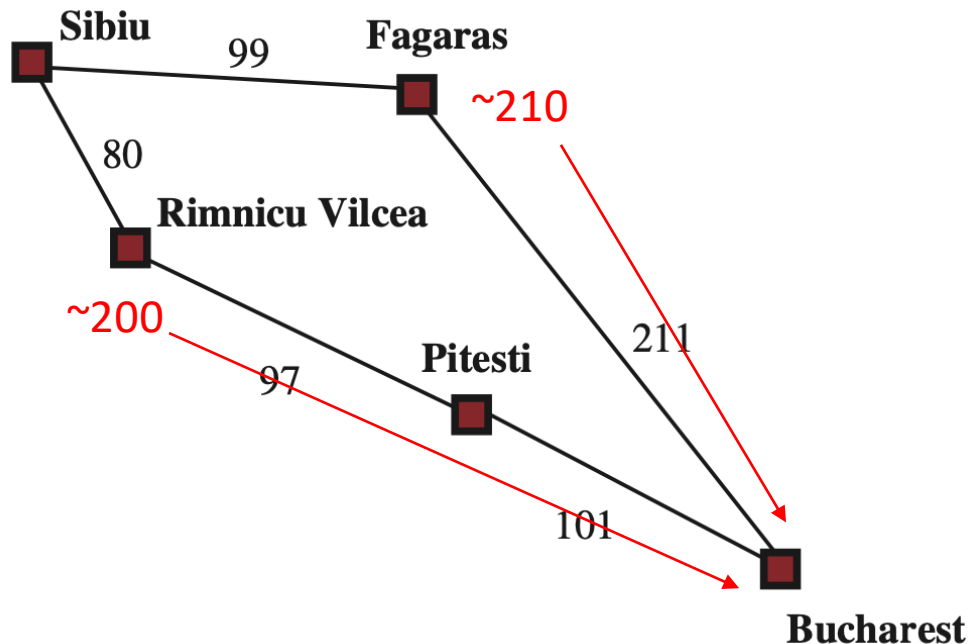
CS2109S: Introduction to AI and Machine Learning

Lecture 3: Local and Adversarial Search

27 August 2026

[PollyV.com/rizki](https://pollyv.com/rizki)

Erratum: Heuristics



Recall that:

- A sequence of actions forms a **path**.
- An **optimal path** has the lowest path cost among all paths.
- An **optimal path cost** is the cost of optimal path.

A heuristic $h(n)$ is an estimate of the optimal path cost from the state at node n to a goal state. A heuristic must be nonnegative and $h(goal)=0$.

Outline

- **Local Search**

- Problem Formulation
- Hill-climbing

- **Adversarial Search**

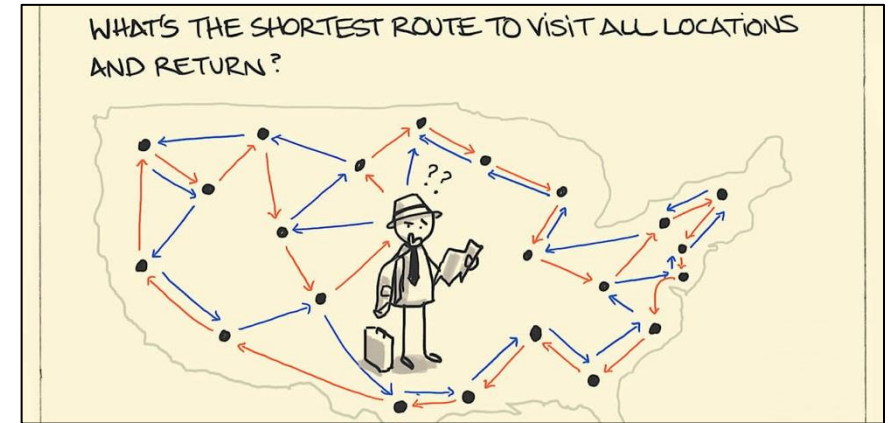
- Problem Formulation
- Minimax
- Alpha-beta Pruning
- “Real-world” Games

Difficult Search & Optimization Problems

- **Combinatorial optimization problems**
 - Example: Travelling Salesperson Problem (TSP)
- **Constraint Satisfaction Problems (CSPs)**
 - Example: Graph coloring problem, scheduling problems
- **Continuous Optimization Problems**
 - Example: non-linear non-convex programming problems

Many of these problems are NP-Hard!

NP-hard: at least as hard as any problem in NP.



Credit: sketchplanations



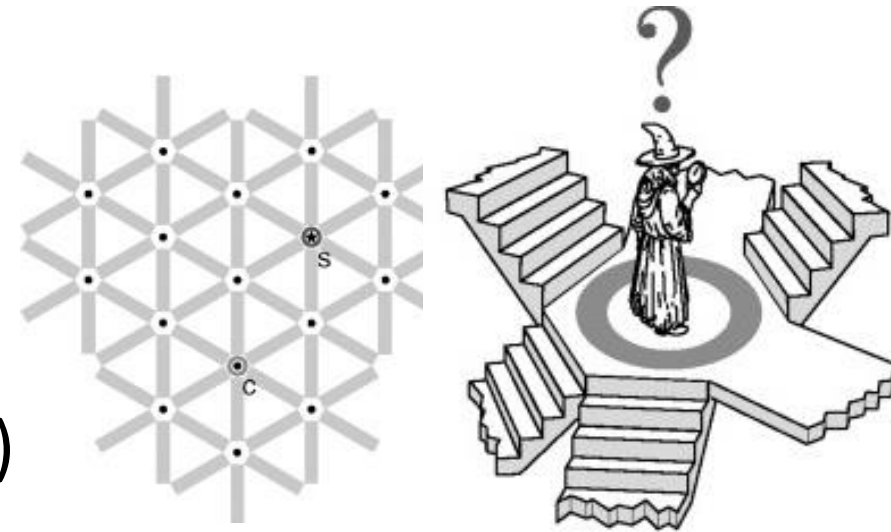
Credit: Classic Computer Science Problems in Java, David Kopec

Previously: Systematic Search

- Traverse the state space in a **systematic** manner
- Typically, **complete** and **optimal** under certain assumptions
 - E.g., BFS, UCS, IDS, A*
- Solving difficult search and optimization problems with this paradigm is **intractable**: not solvable within a reasonable amount of time

Local Search

- Local search algorithms operate by searching **from a start state to neighbouring states**. It does not keep track of
 - the paths
 - the set of states that have been reached.
- A solution is a state.
- It's not systematic: may never explore a region of state space (which may contain the solution)
 - Thus, typically **incomplete** and **suboptimal**.
- Benefits:
 - Use very little memory.
 - Can find a **reasonable** solution in problems with **large** or **infinite** state space.



Holger H. Hoos, Thomas Stützle, in
[Stochastic Local Search](#), 2005

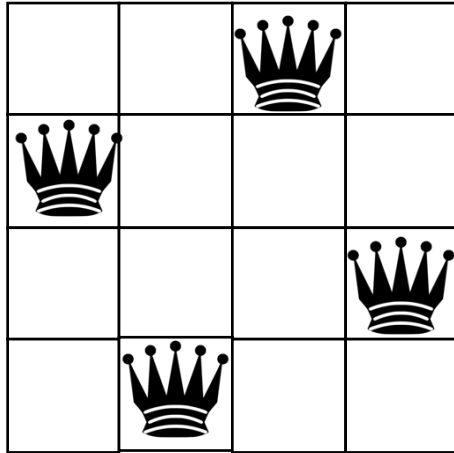
Problem Formulation: Local Search

- **States:** A set of possible states that the environment could be in. We call this **state-space**. Local search uses a broader definition: it represents different configurations or candidate solutions.
 - **Initial State:** The starting configuration
 - **Goal Test / ~~State(s)~~:** In local search, a solution is a state. In some problems, we don't even have to define a goal test.
 - **Actions**
 - **Transition model**
 - **Action cost function**
- Successor Function:** A function that generates neighboring states by applying modifications to the current state.

In some cases, we don't have to define these three explicitly.

Generated states must be valid according to the actions, transition model, and action cost function.

Example: N-Queens



States

Configurations of queens on the board

Initial state

Random placement of queens, one queen on each column

Goal test

No two queens threaten each other,
i.e., no two queens share the same row, column, or diagonal

Successor function

Move a queen to a different row within the same column

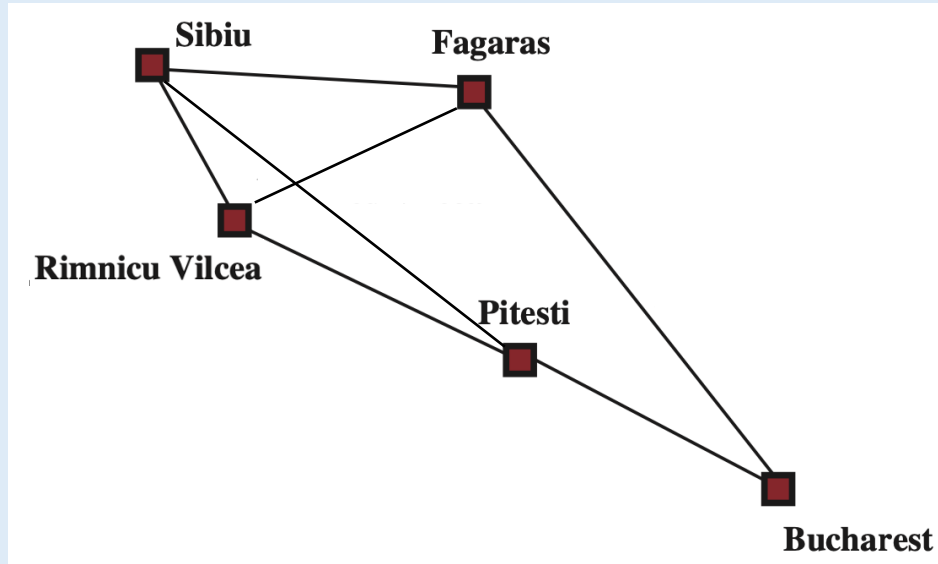
Example: Minimizing a Function

Find an **integer** x that minimizes a “black box” function $f(x)$

States	All possible integers
Initial state	Random integer
Goal test	None (optimization problem)
Successor function	Make change to x , either by increasing or decreasing it by 1

PollEverywhere

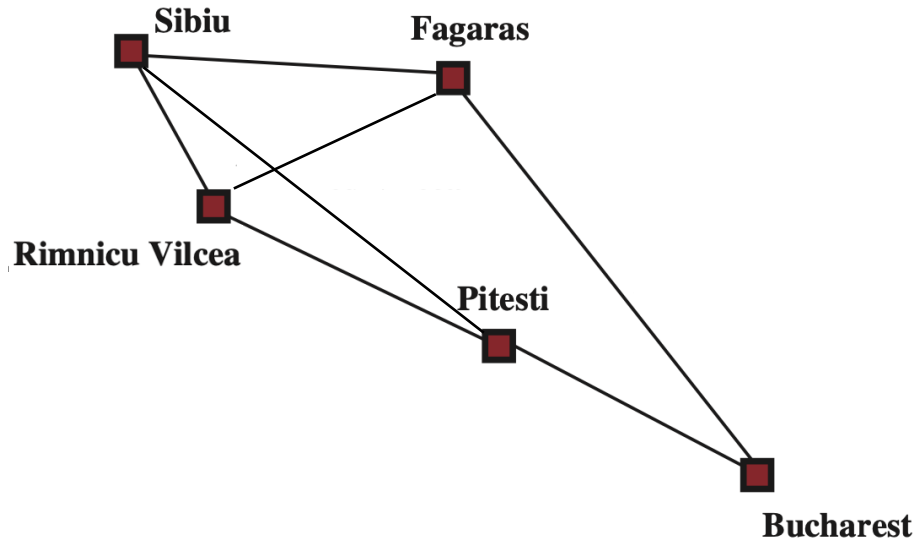
PollEV.com/rizki



In local search, a solution is a state rather than the path taken to reach it. Can local search be used to potentially find a good solution for shortest path problems?

- A. Yes
- B. No
- C. It depends

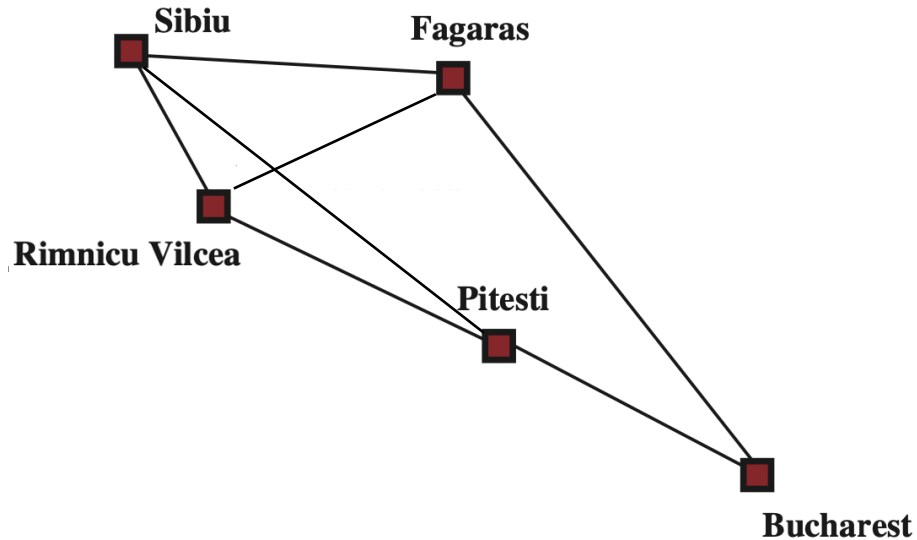
PollEverywhere



In local search, a solution is a state rather than the path taken to reach it. Can local search be used to potentially find a good solution for shortest path problems?

- A. Yes
- B. No
- C. It depends

Example: Path Finding



States

Initial state

Goal test

Successor function

Paths from Sibiu to Bucharest

Random path from Sibiu to Bucharest

None

Replace a sub-path between any arbitrary two cities in the current path with other valid paths

Example:

- Sibiu – Fagaras – Rimnicu Vilcea – Pitesti – Bucharest



- Sibiu – Fagaras – Bucharest
- Sibiu – Fagaras – Rimnicu Vilcea – Sibiu – Pitesti – Bucharest

Evaluation Function

An **evaluation function** (also known as a **fitness function** or **objective function**) is a mathematical function used to assess the quality or desirability of a state.

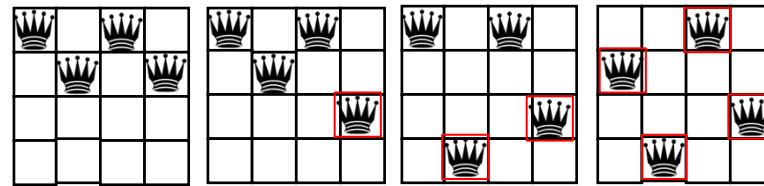
The function takes in a state as an argument and produces a number.

```
value = eval(state)
```

Depending on the problem and the algorithm, we may want to either minimize or maximize this function.

Evaluation Function - Examples

- In the **n-queens problem**, an evaluation function can be the number of "safe" queens, i.e., queens not attacking each other in the same row, column, or diagonal. The goal is to maximize this number.



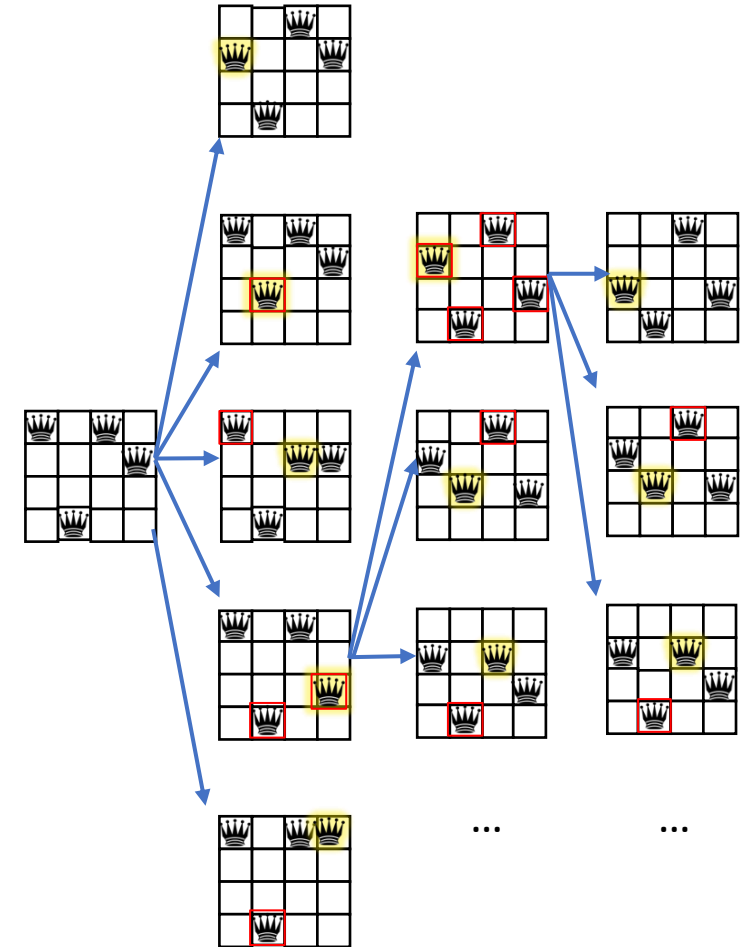
- In the problem of **minimizing a function**, the evaluation function is the (objective) function itself, i.e., $f(x)$. The goal is to minimize it.

Note: A minimization objective $f(x)$ can be transformed into a maximization objective by defining a new objective: $\hat{f}(x) = -f(x)$, and vice versa.

Hill climbing algorithm

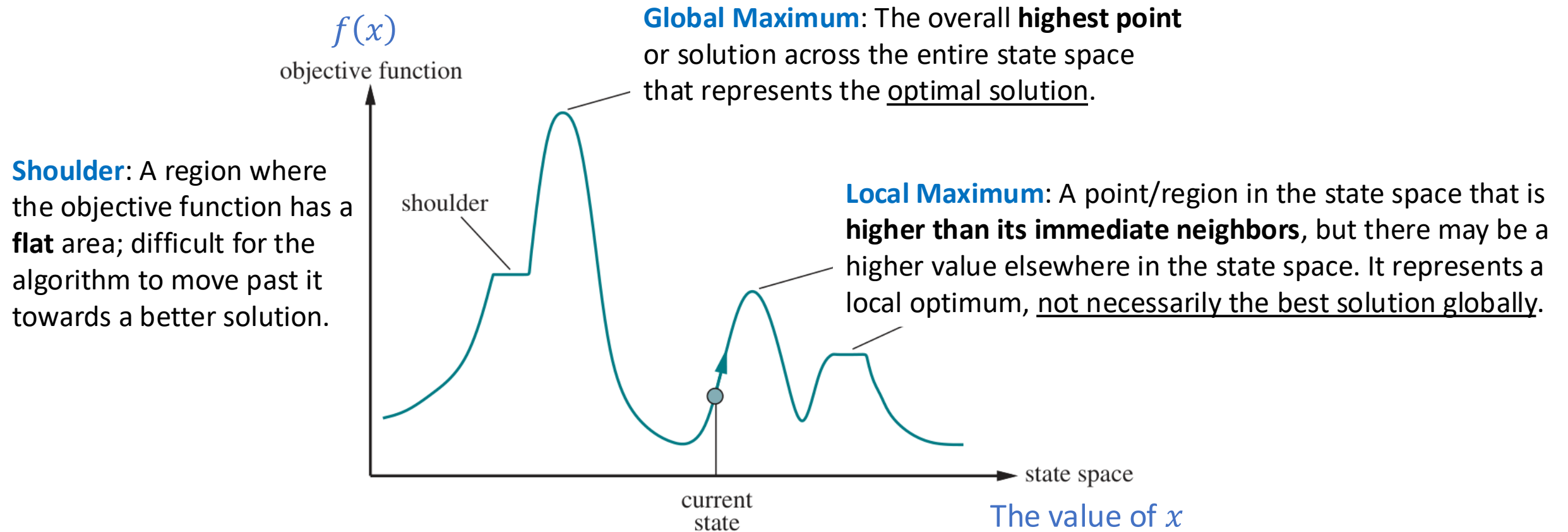
(Steepest Ascent Search, Greedy Local Search)

```
current_state = initial_state
while True:
    best_successor = a highest-eval successor state of current_state
    if eval(best_successor) <= eval(current_state):
        return current_state
    current_state = best_successor
```



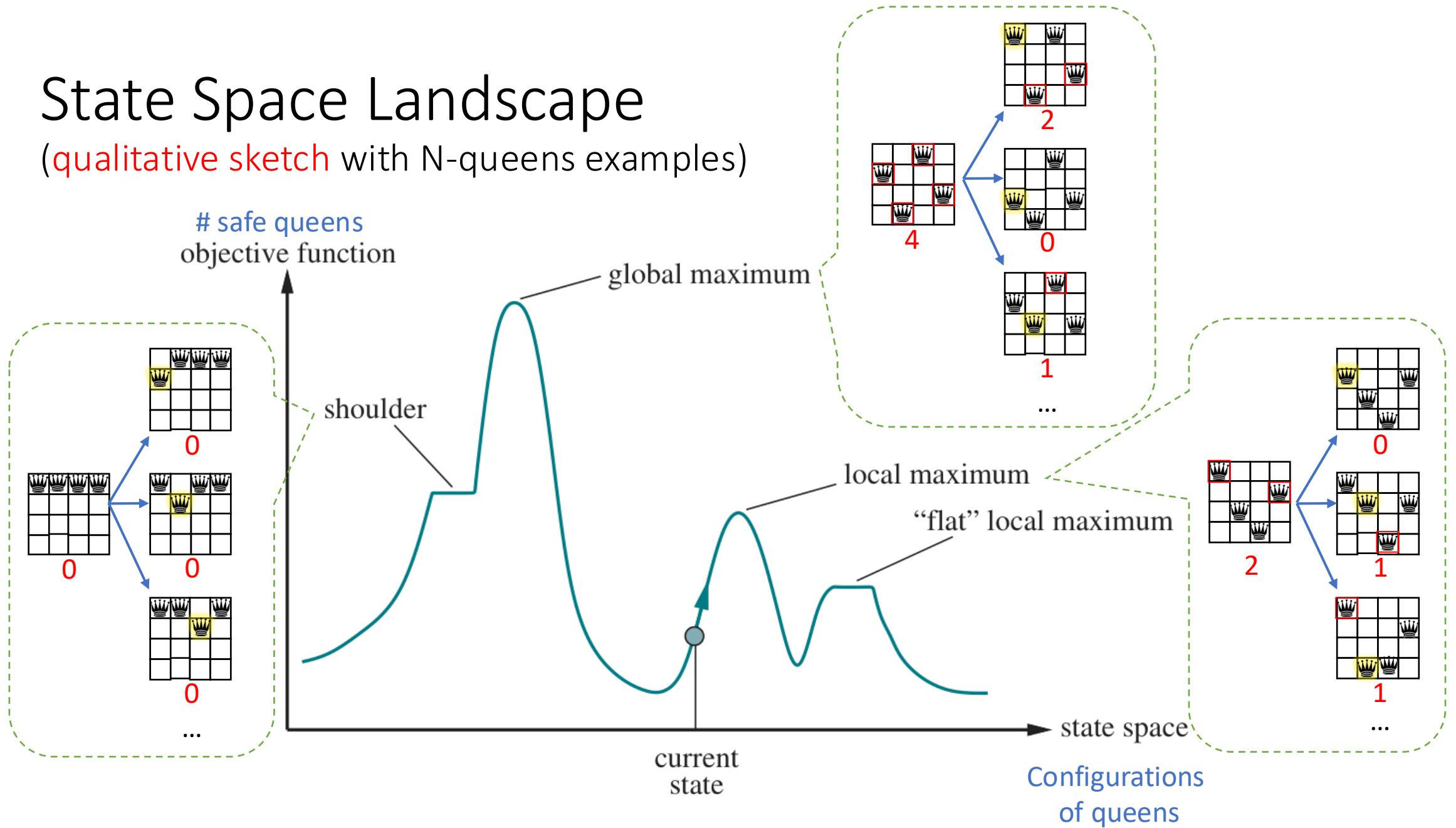
State Space Landscape

(black-box function maximization)



State Space Landscape

(qualitative sketch with N-queens examples)



Real-world uses

- TimberWolf VLSI Chip Placement
 - Simulated annealing (hill climbing, but occasionally pick bad neighbor)
- NASA Spacecraft / Antenna Design
 - Local search
- SAT Solving and Constraint Satisfaction
 - Hill climbing variants
- Timetabling, vehicle routing, etc

Break

A BRIEF HISTORY OF CHESS



Outline

- Local Search
 - Problem Formulation
 - Hill-climbing
- **Adversarial Search**
 - Problem Formulation
 - Minimax
 - Alpha-beta Pruning
 - “Real-world” Games

Multi-agent Problems

Multi-agent problems involve **multiple agents** interacting within an environment, where each agent has its own goal(s) and decision-making process. The problems could be:

- **Competitive:** Agents have opposing goals
 - Example: chess, go
- **Cooperative:** Agents work together towards a common goal
 - Example: robotic swarms
- **Mixed-Motive:** Some cooperation and some competition
 - Example: auctions, the Prisoner's Dilemma
- ...

Competitive Multi-agent Problems

Conflicting Goals: Agents have opposing goals, and one agent's success might directly impact the failure of the other.

Example:

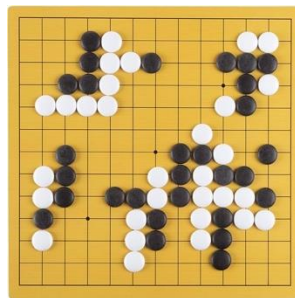
- **Adversarial Games:** Players (agents) compete against each other.
 - **Two-player zero-sum games:** one player's gain is the other player's loss.
Example: chess, go, tic-tac-toe, 2-player poker.

• ...

• ...



Credit: chess.com



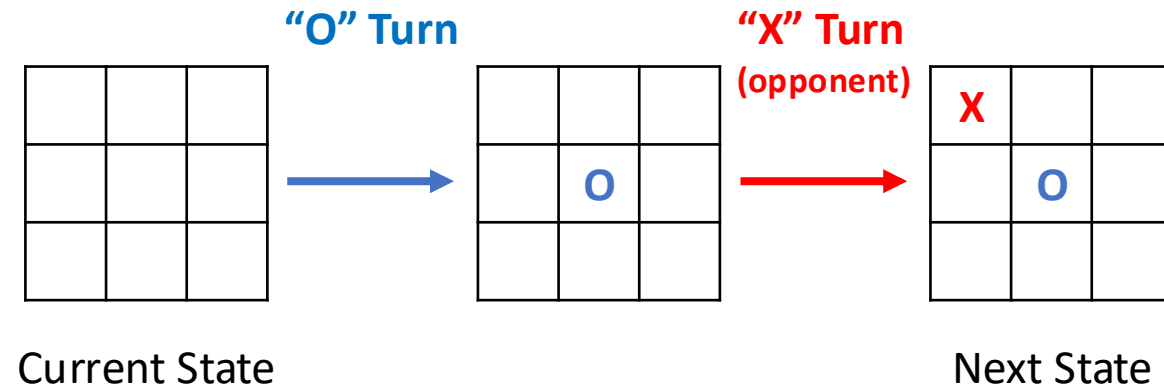
Credit: Amazon.sg



Credit: Wikipedia

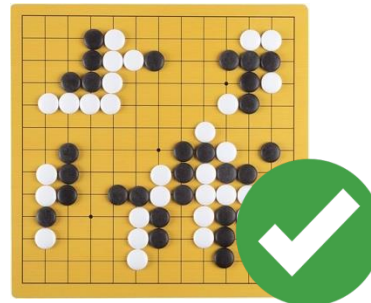
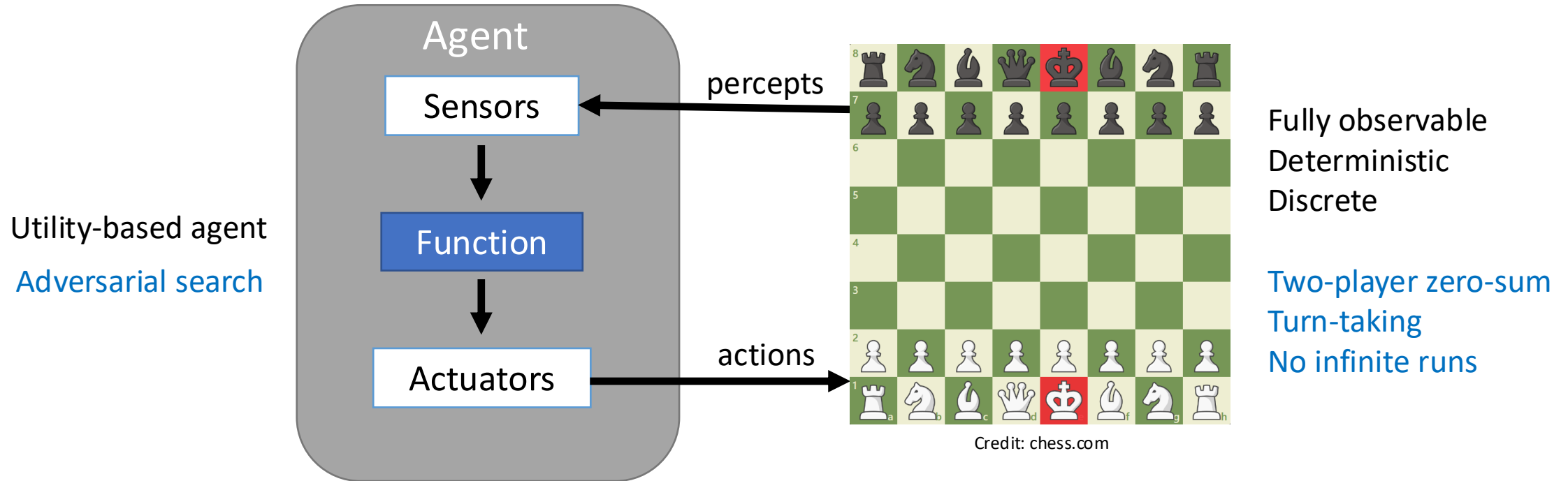
Turn-Taking Games

The next state of the environment depends not only on our action in the current turn but also on the action of other agents in the next turn.



Designing an Agent

for competitive multi-agent problems (i.e., ^{“classical”}adversarial games)



Credit: Amazon.sg



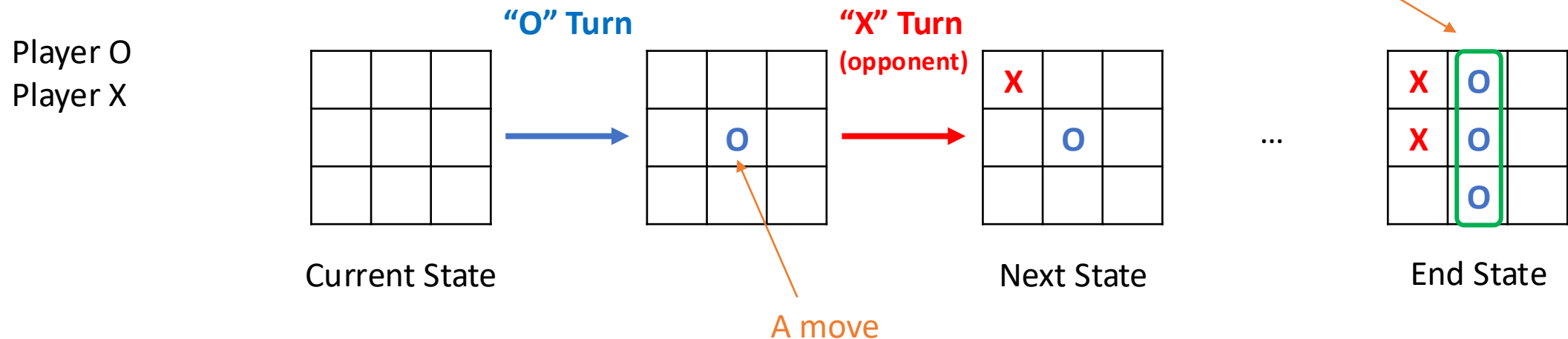
Credit: Wikipedia



Credit: Toys R Us

Games – Terminology

- **Player** (Agent): An entity making decisions or taking actions in a game.
- **Turn**: A phase where a player makes a move, after which control passes to the next player.
- **Move** (Action): A decision or action by a player that changes the game state.
- **End State** (Terminal State): A state where the game ends, with no further moves possible.
- **Winning Condition**: Criteria that define when a player wins the game.
- **Optimal**: Get the best/highest outcome. (not about minimizing path cost)



Games – Game Tree

A game tree is a graphical representation of all possible states and moves in a game.

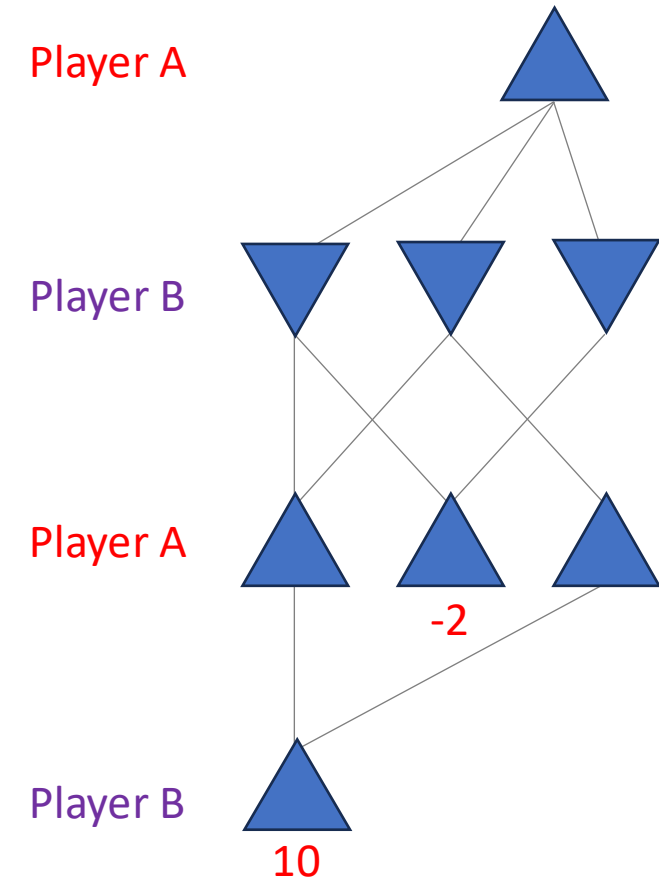
In this diagram:

- **Nodes** indicate possible game states.
- **Edges** show legal moves or transitions between states.

The **outcome of the game** (such as monetary payoff, win, loss, or draw) is displayed beneath each terminal node (end state).

The outcome is from the perspective of Player A.

- ▲ Game states where it is **Player A's** turn
- ▼ Game states where it is **Player B's** turn



Recall:

Problem Formulation in Systematic Search

- **States**
- **Initial State**
- **Goal Test / State(s)**

- **Actions**
- **Transition**
- **Action cost function**

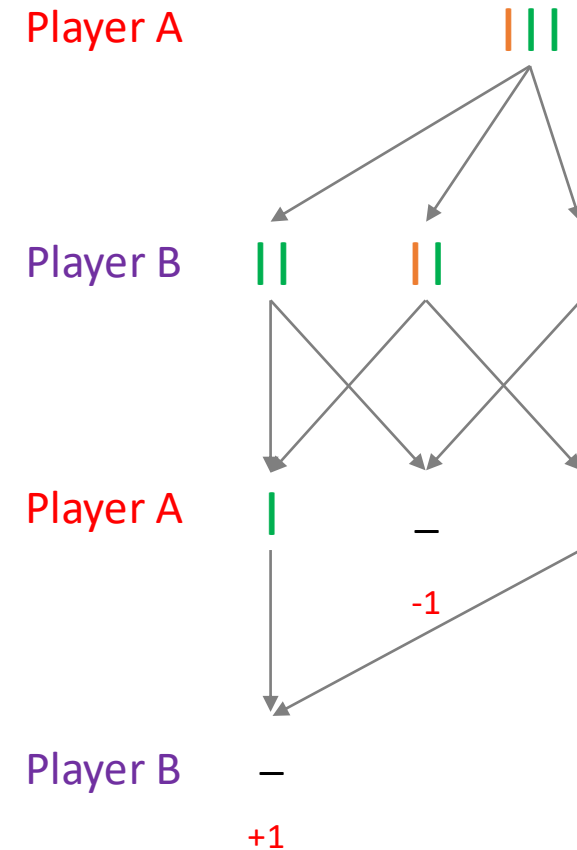
Problem Formulation in Adversarial Search

- **States**
- **Initial State**
- **Terminal States** : states where the game ends, such as win/lose/draw states.
 - This set of states could be defined as a function.
- **Actions**
- **Transition**
- **Utility Function**: output the value of a state from the perspective of our agent.
 - Our agent (player A) wants to maximize the utility.
 - In games such as Chess or tic-tac-toe, the outcome for player A is a win, loss, or draw.
 - Assign utility values of +1, -1, and 0, respectively, to represent these outcomes.
 - In other games such as Go with territory scoring, the outcome for player A is some number.
 - Assign integer/real-valued utility values to the outcomes.
 - The utility of non-terminal states is typically not known in advance and must be computed.

Example – Sticks

- Two piles of sticks (**green** and **orange**) are given
- Player can take any number of sticks from a single pile
- The player who cannot make any move loses

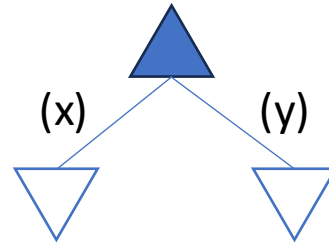
States	Configurations of piles
Initial state	A number of green sticks and orange sticks
Terminal states	No stick (no move available)
Actions	Take 1,2,... sticks from one of the piles
Transition	Remove the sticks from the pile
Utility function	+1 for win, -1 for lose • Win if Player B can't make any move • Lose if Player A can't make any move



Q: Which move would you choose?

Assume you are Player A and the game is in some state. Which move (x) or (y)?

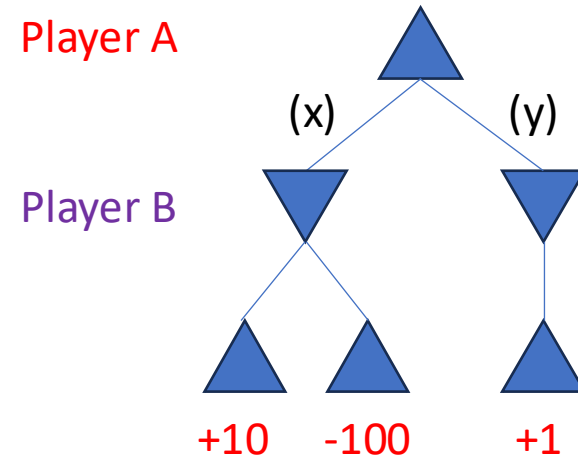
Player A



Q: Which move would you choose?

Assume you are Player A and the game is in some state. Which move (x) or (y)?

What about now?



Minimax – Overview

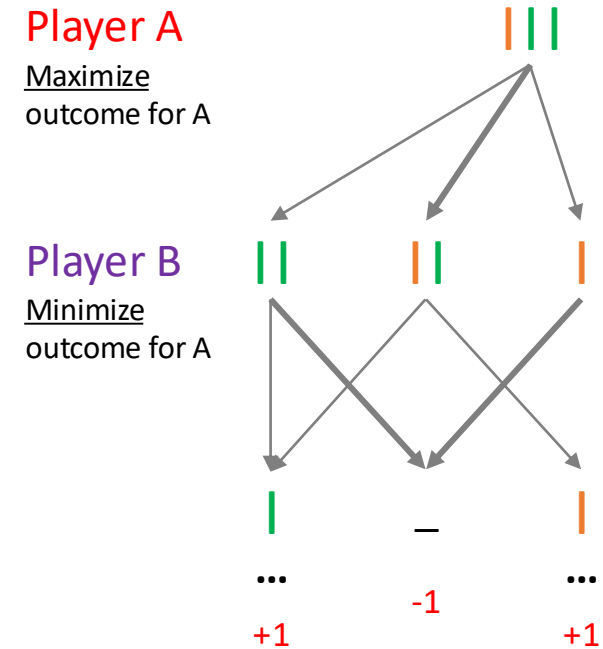
Algorithm for **two-player zero-sum** game.

Core assumption: all players play optimally.

View the outcome from the perspective of Player A.

- Player A tries to **maximize** its outcome.
- Player B tries to **minimize** the outcome of Player A, i.e., maximize their own outcome.

Minimax algorithm computes the outcome of the game by **evaluating backwards from the leaves**.



Minimax – Functions

Expand function **expand**(state)

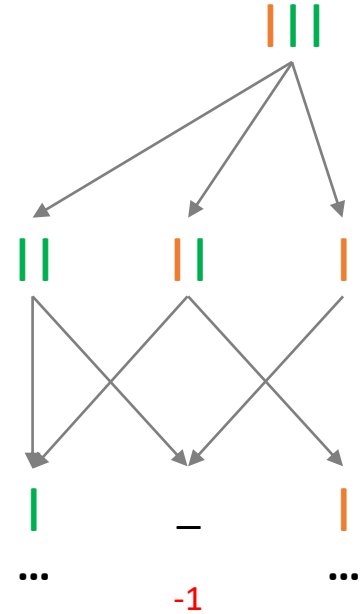
- For each **action**, compute: **next_state** = **transition**(state, **action**)
- Returns a set of (**action**, **next_state**) pairs.

Terminal function **is_terminal**(state)

- Returns true/false if the state is a terminal state.

Utility function: **utility**(state)

- Returns a score for the state from the point of view of Player A.



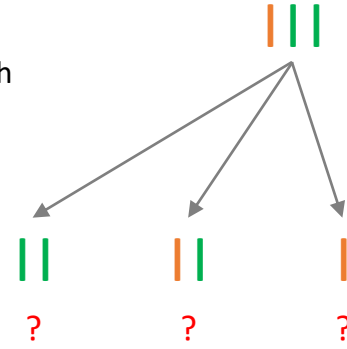
Minimax

```
def minimax(state):  
    (action, v) = max_value(state)  
    return action
```

Note: We assume here that **max_value** also returns the action to which the value v corresponds to. We do not write this explicitly in the following pseudo-code.

Player A

Choose action with
max value for A



Minimax

```
def minimax(state):  
    (action, v) = max_value(state)  
    return action
```

```
def max_value(state):  
    if is_terminal(state): return utility(state)  
    v = -∞ // smallest value possible  
    for next_state in expand(state):  
        v = max(v, minimum value for player A in the next_state)  
    return v
```

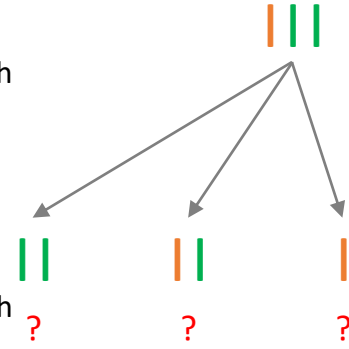
Assumes opponent play **optimally**:
trying to **minimize** player A value

Player A

Choose action with
max value for A

Player B

Choose action with
min value for A

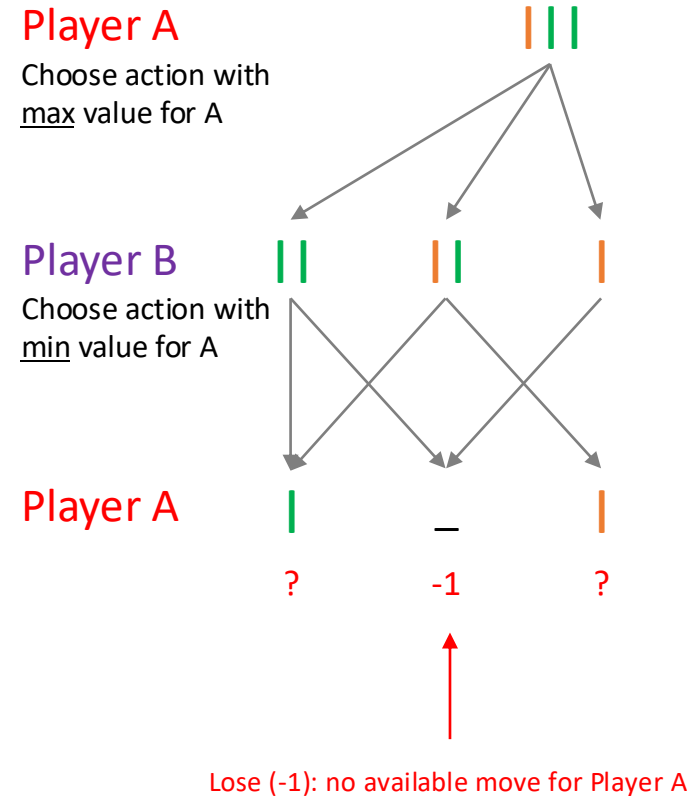


Minimax

```
def minimax(state):  
    (action, v) = max_value(state)  
    return action
```

```
def max_value(state):
    if is_terminal(state): return utility(state)
    v = -∞ // smallest value possible
    for next_state in expand(state):
        v = max(v, min_value(next_state))
    return v
```

```
def min_value(state):
    if is_terminal(state): return utility(state)
    v =  $\infty$  // largest value possible
    for next_state in expand(state):
        v = min(v, max_value(next_state))
    return v
```



Minimax – Demo

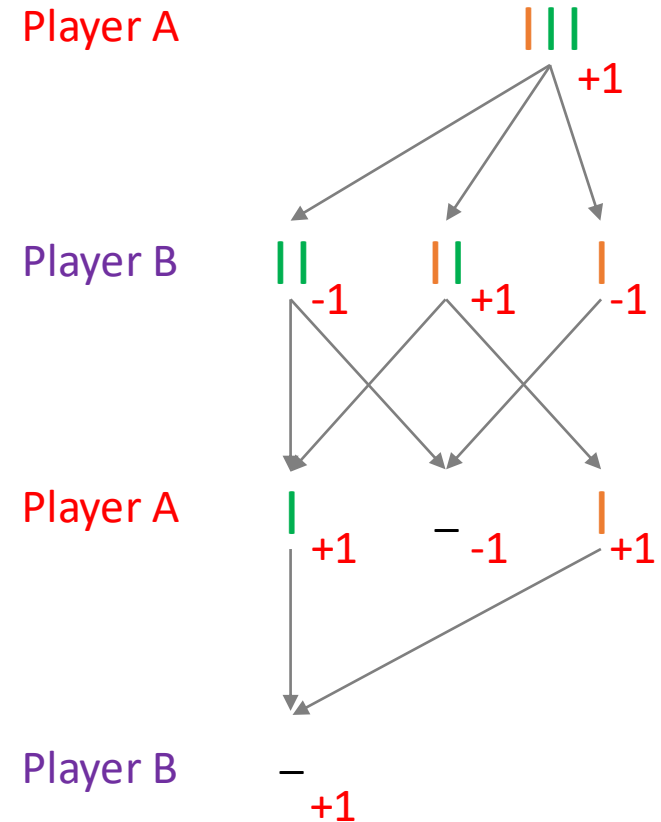
```
def minimax(state):
    (action, v) = max_value(state)
    return action

def max_value(state):
    if is_terminal(state): return utility(state)
    v = -∞ // smallest value possible
    for next_state in expand(state):
        v = max(v, min_value(next_state))
    return v

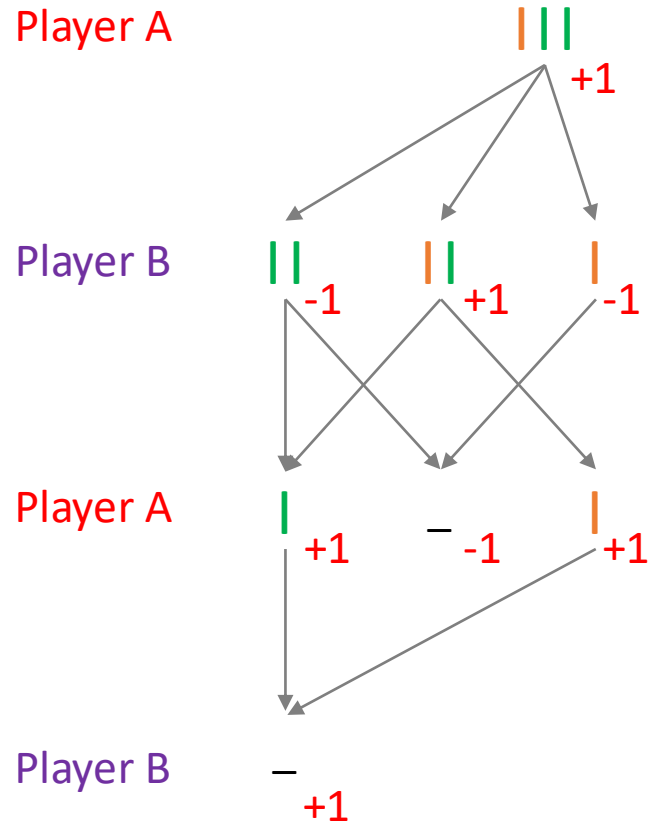
def min_value(state):
    if is_terminal(state): return utility(state)
    v = ∞ // largest value possible
    for next_state in expand(state):
        v = min(v, max_value(next_state))
    return v
```

Player A: maximize

Player B: minimize



Minimax – Analysis



Time Complexity?

Exponential w.r.t. maximum depth: $O(b^m)$

Branching factor b , max-depth m

Space Complexity?

Polynomial

Complete?

Yes, if tree is finite

Optimal?

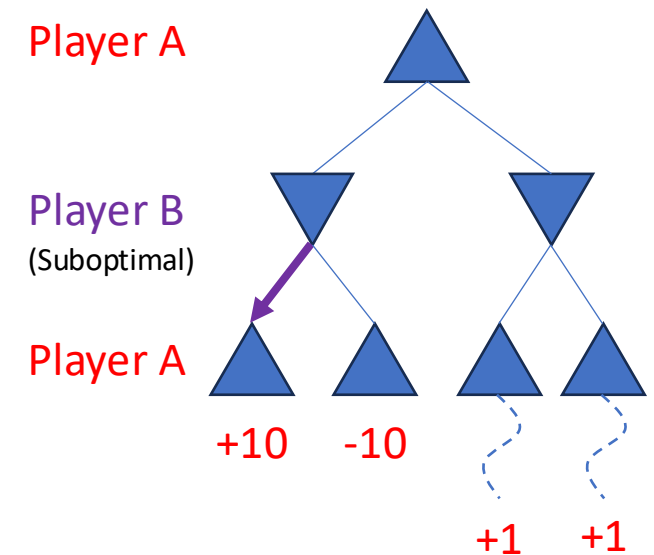
Yes, against optimal opponent

Minimax – Optimality

Theorem 1: In a finite, two-player, zero-sum game with perfect information, the minimax algorithm finds the optimal strategy for each player. If both play optimally, each gets the best outcome they possibly can.

Theorem 2: If you play the optimal (minimax) strategy and your opponent makes mistakes (i.e., play sub-optimally), you'll never do worse than you would against a perfect opponent — a weaker opponent can only help you.

Minimax keeps you safe by assuming your opponent is optimal. But against a mistake-prone opponent, a bolder strategy might exploit those errors and win you even more.



Q: Which move would you choose?

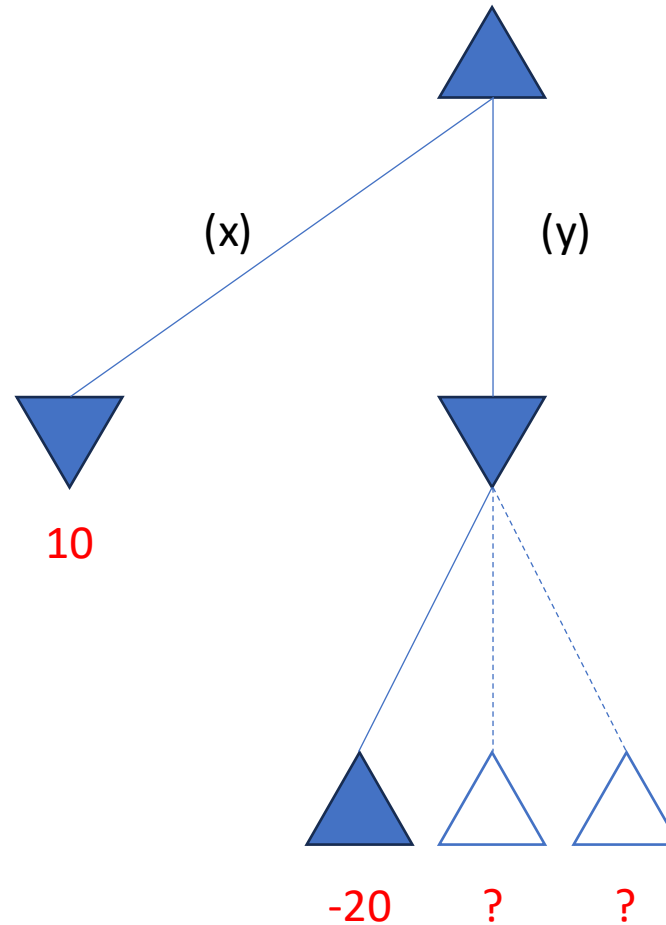
Assume you are
Player A.

Player A
Maximize

Opponent plays
optimally.

Do you choose
move (x) or (y)?

Player B
Minimize



The Problem with Minimax

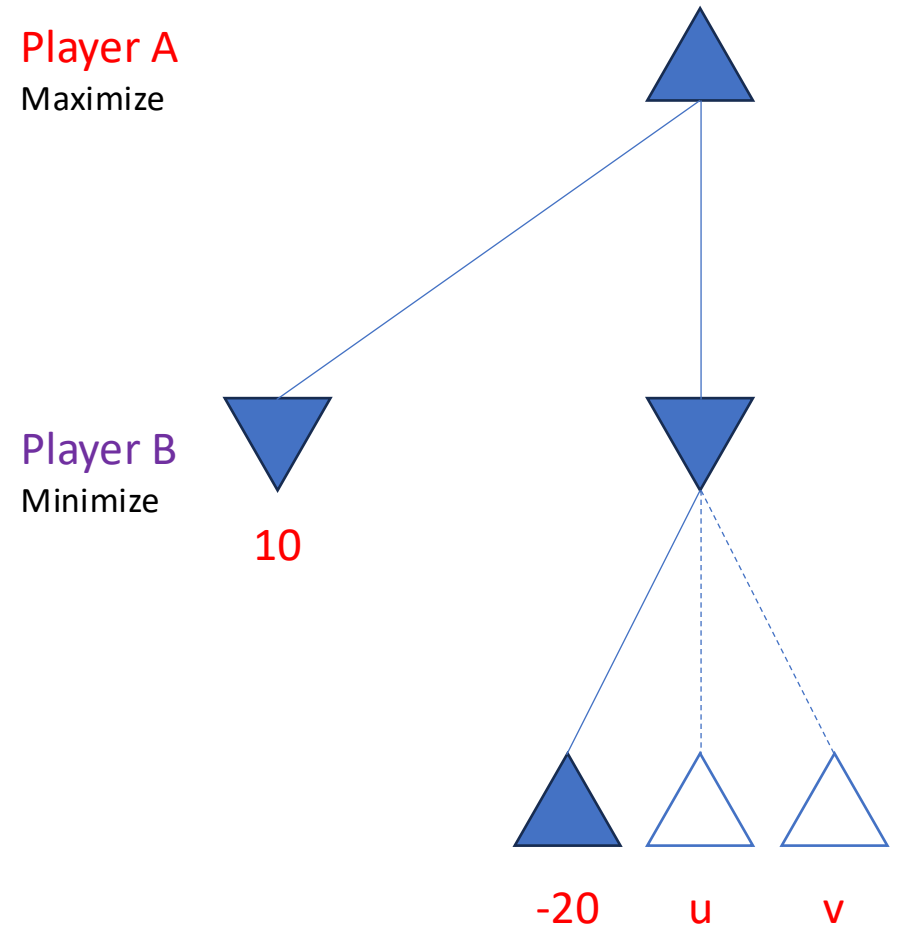
Minimax explores the entire game tree.

However, evaluating a node sometimes becomes unnecessary, as it won't alter the decision.

Logical view:

$$\text{max}(10, \text{min}(-20, u, v)) = 10$$

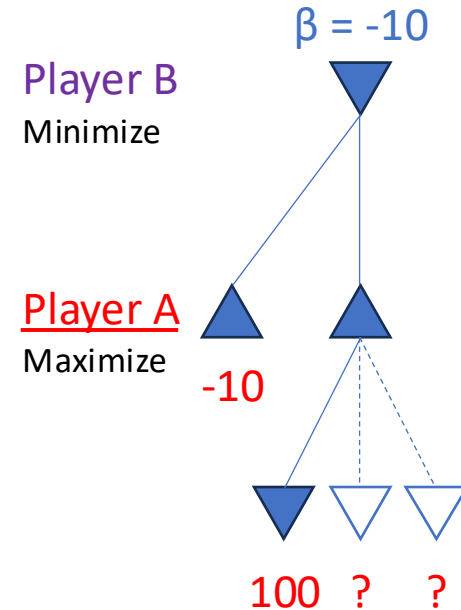
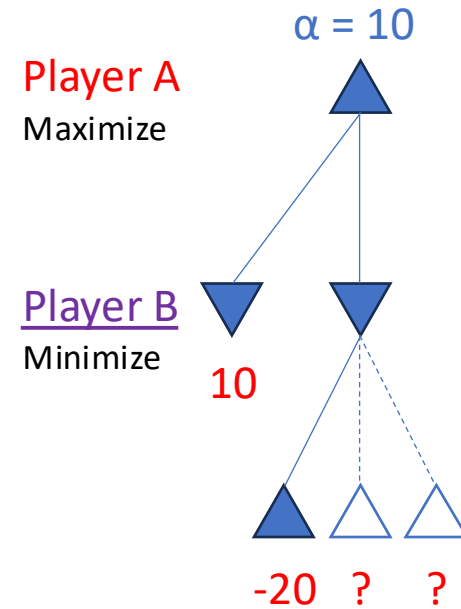
for all u, v .



Alpha-Beta Pruning – Overview

Optimized version of the Minimax algorithm. It produces the same result but evaluates fewer nodes, making the search faster.

- While searching, we keep track of two values:
 - Player A: highest value for A so far (called α)
 - Player B: lowest value for A so far (called β)
- Prune unnecessary nodes
 - **Player B turn:** If one move already looks **bad for Player A** (value $\leq \alpha$), stop checking the rest — B will only make it worse, and A won't pick this path anyway.
 - **Player A turn:** If one move already looks **great for Player A** (value $\geq \beta$), stop checking the rest — A will only make it better, and B won't allow this path anyway.



```
def alpha_beta_search(state):  
    (action, v) = max_value(state,  $-\infty$ ,  $\infty$ )  
    return action
```

Alpha-Beta Pruning

➡ **Player A**
Choose action with
max value for A

$$\alpha = -\infty$$
$$\beta = \infty$$



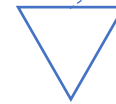
Best outcome so far

Player A: highest value for A so far (α)

Player B: lowest value for A so far (β)

At the beginning, both players are **pessimistic** about the best outcome:

- Worst-case for A: $-\infty$ (A loses big)
- Worst-case for B: ∞ (A wins big)



?



?

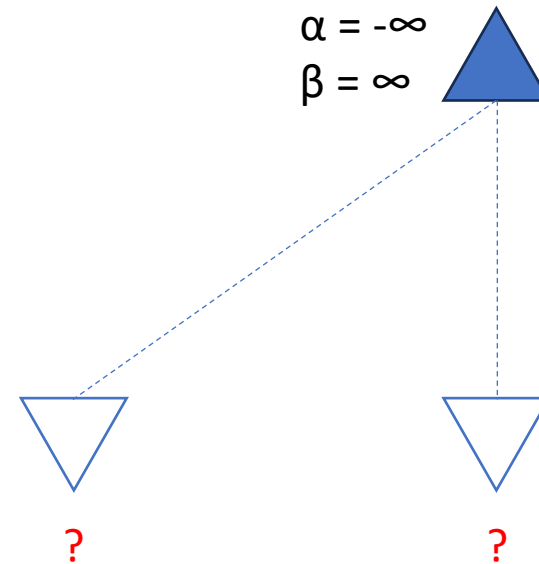
```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action
```

```
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v
```

Alpha-Beta Pruning

➡ **Player A**
Choose action with
max value for A

Player B
Choose action with
min value for A



Best outcome so far
Player A: highest value for A so far (α)
Player B: lowest value for A so far (β)

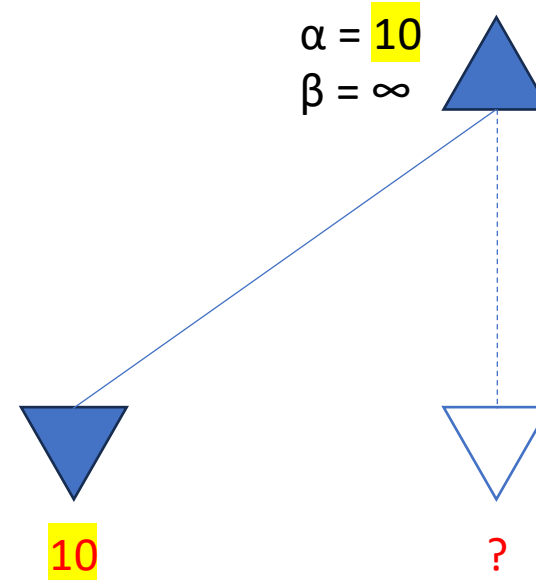
```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action
```

```
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v
```

Alpha-Beta Pruning

➡ **Player A**
Choose action with
max value for A

Player B
Choose action with
min value for A



Best outcome so far
Player A: highest value for A so far (α)
Player B: lowest value for A so far (β)

Outcome for A in this node is better
than best so far ($-\infty$), update α

```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action
```

```
def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v
```

```
def min_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

Alpha-Beta Pruning

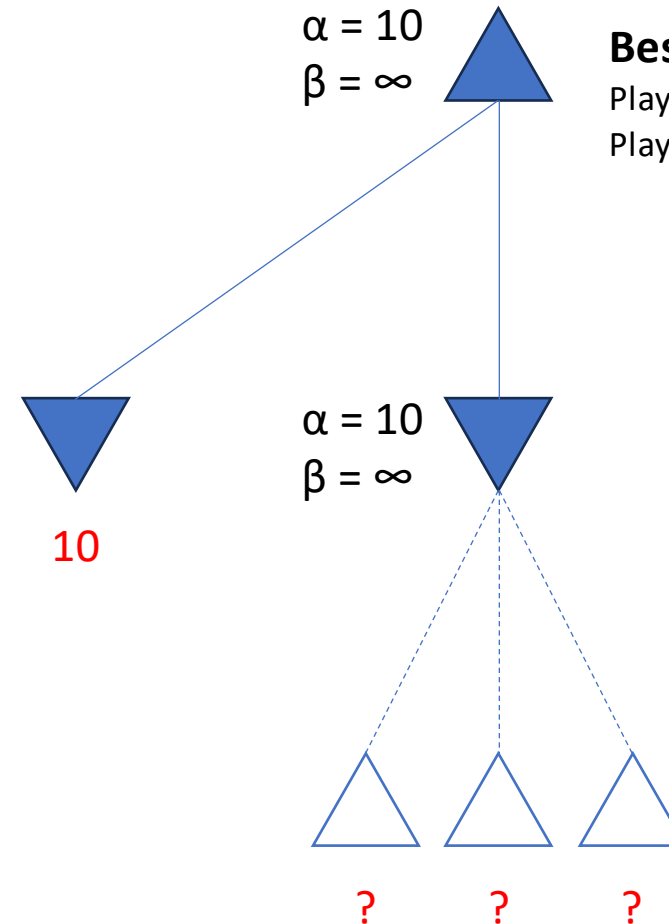
Player A

Choose action with
max value for A

Player B

Choose action with
min value for A

Player A



Best outcome so far

Player A: highest value for A so far (α)

Player B: lowest value for A so far (β)

```
def alpha_beta_search(state):
```

```
    (action, v) = max_value(state, -∞, ∞)
```

```
    return action
```

```
def max_value(state, α, β):
```

```
    if is_terminal(state): return utility(state)
```

```
    v = -∞
```

```
    for next_state in expand(state):
```

```
        v = max(v, min_value(next_state, α, β))
```

```
        α = max(α, v)
```

```
        if v >= β: return v
```

```
    return v
```

```
def min_value(state, α, β):
```

```
    if is_terminal(state): return utility(state)
```

```
    v = ∞
```

```
    for next_state in expand(state):
```

```
        v = min(v, max_value(next_state, α, β))
```

```
        β = min(β, v)
```

```
        if v <= α: return v
```

```
    return v
```

Alpha-Beta Pruning

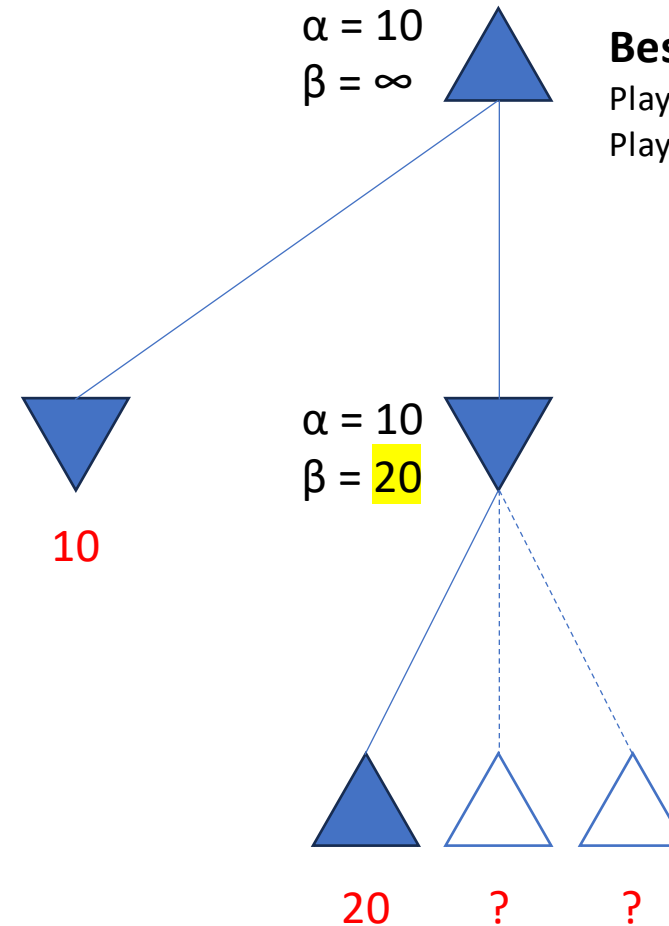
Player A

Choose action with
max value for A

Player B

Choose action with
min value for A

Player A



Best outcome so far

Player A: highest value for A so far (α)

Player B: lowest value for A so far (β)

Outcome for B in this node is better
than best so far (∞), update β

```
def alpha_beta_search(state):
```

```
    (action, v) = max_value(state, -∞, ∞)
```

```
    return action
```

```
def max_value(state, α, β):
```

```
    if is_terminal(state): return utility(state)
```

```
    v = -∞
```

```
    for next_state in expand(state):
```

```
        v = max(v, min_value(next_state, α, β))
```

```
        α = max(α, v)
```

```
        if v >= β: return v
```

```
    return v
```

```
def min_value(state, α, β):
```

```
    if is_terminal(state): return utility(state)
```

```
    v = ∞
```

```
    for next_state in expand(state):
```

```
        v = min(v, max_value(next_state, α, β))
```

```
        β = min(β, v)
```

```
        if v <= α: return v
```

```
    return v
```

Alpha-Beta Pruning

Player A

Choose action with
max value for A

Player B

Choose action with
min value for A

Player A

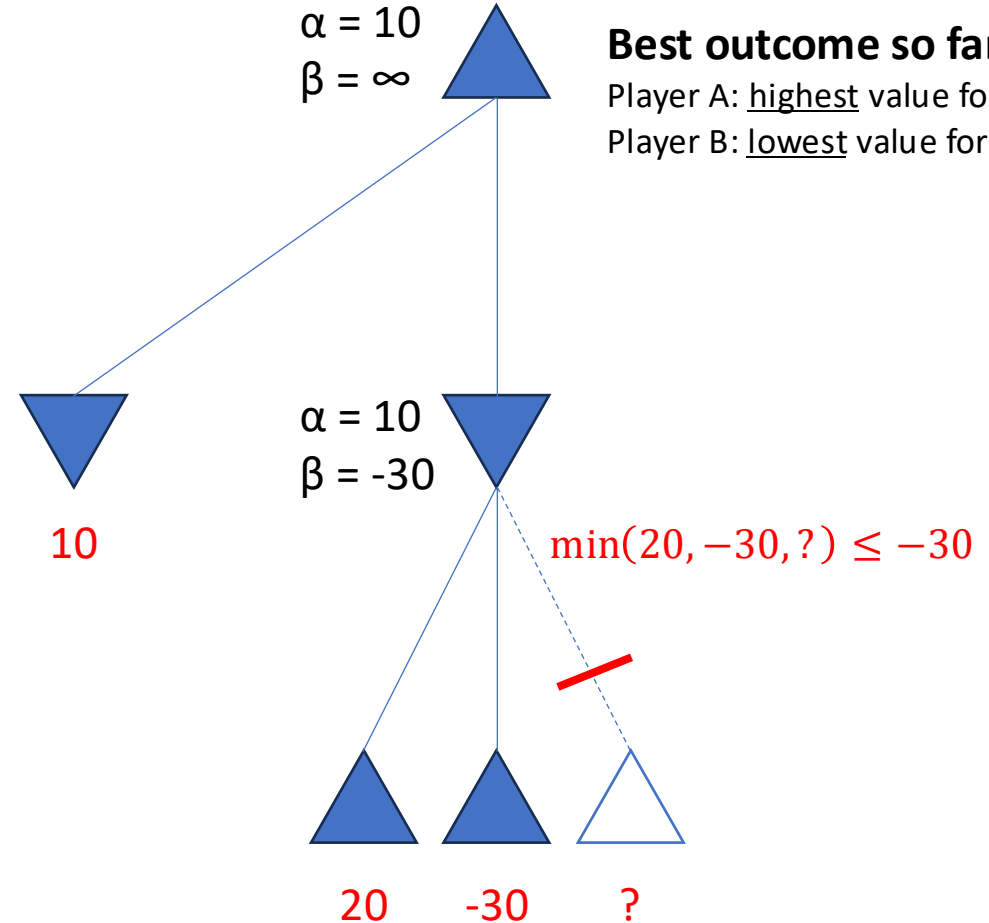
$\max(10, \min(\dots)) = 10$

$\alpha = 10$
 $\beta = \infty$

Best outcome so far

Player A: highest value for A so far (α)

Player B: lowest value for A so far (β)

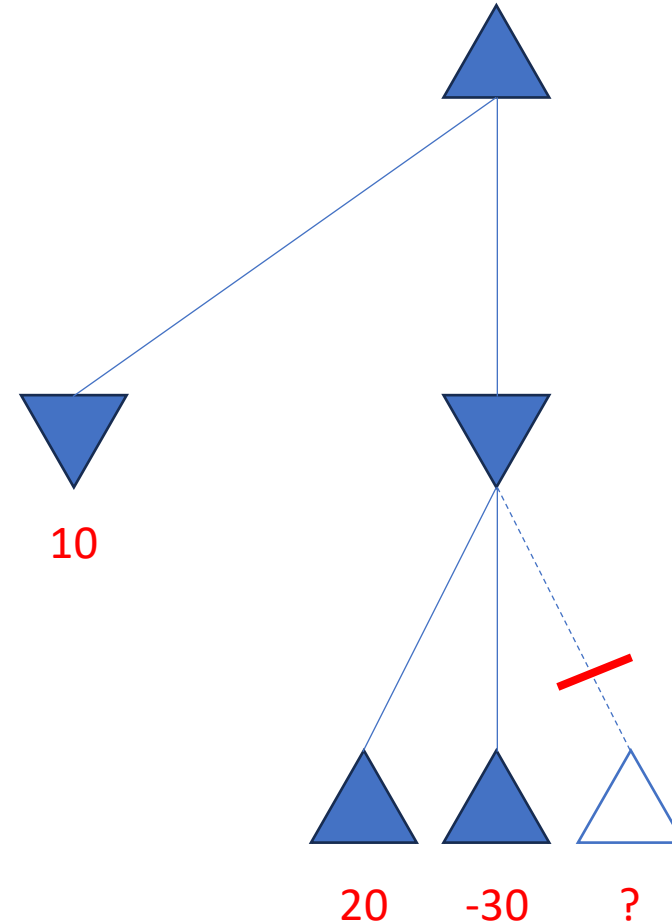


Worse than the best outcome for Player A so far (10)
Player A in the root node will not pick the 2nd action⁴⁹

Alpha-Beta Pruning – Analysis

Theorem: Alpha-beta pruning does not alter the minimax **value** and the **optimal move** computed for the root node; it only reduces the number of nodes evaluated.

Logical view: $\max(10, \min(20, -30, ?)) = 10$

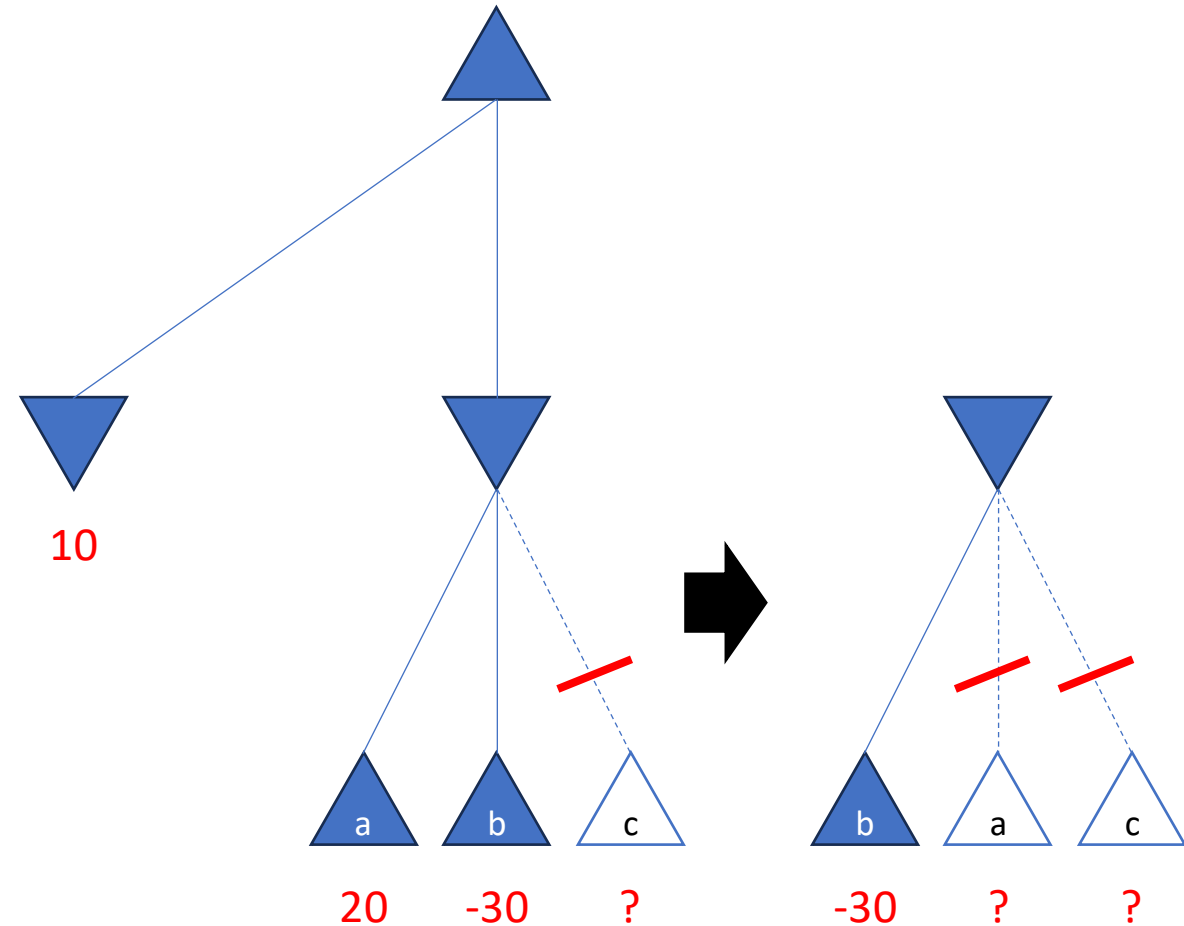


Alpha-Beta Pruning – Analysis

Theorem: Alpha-beta pruning does not alter the minimax **value** and the **optimal move** computed for the root node; it only reduces the number of nodes evaluated.

Logical view: $\max(10, \min(20, -30, ?)) = 10$

Making full use of alpha-beta pruning requires **meta-reasoning**: reasoning about which computations to do first.



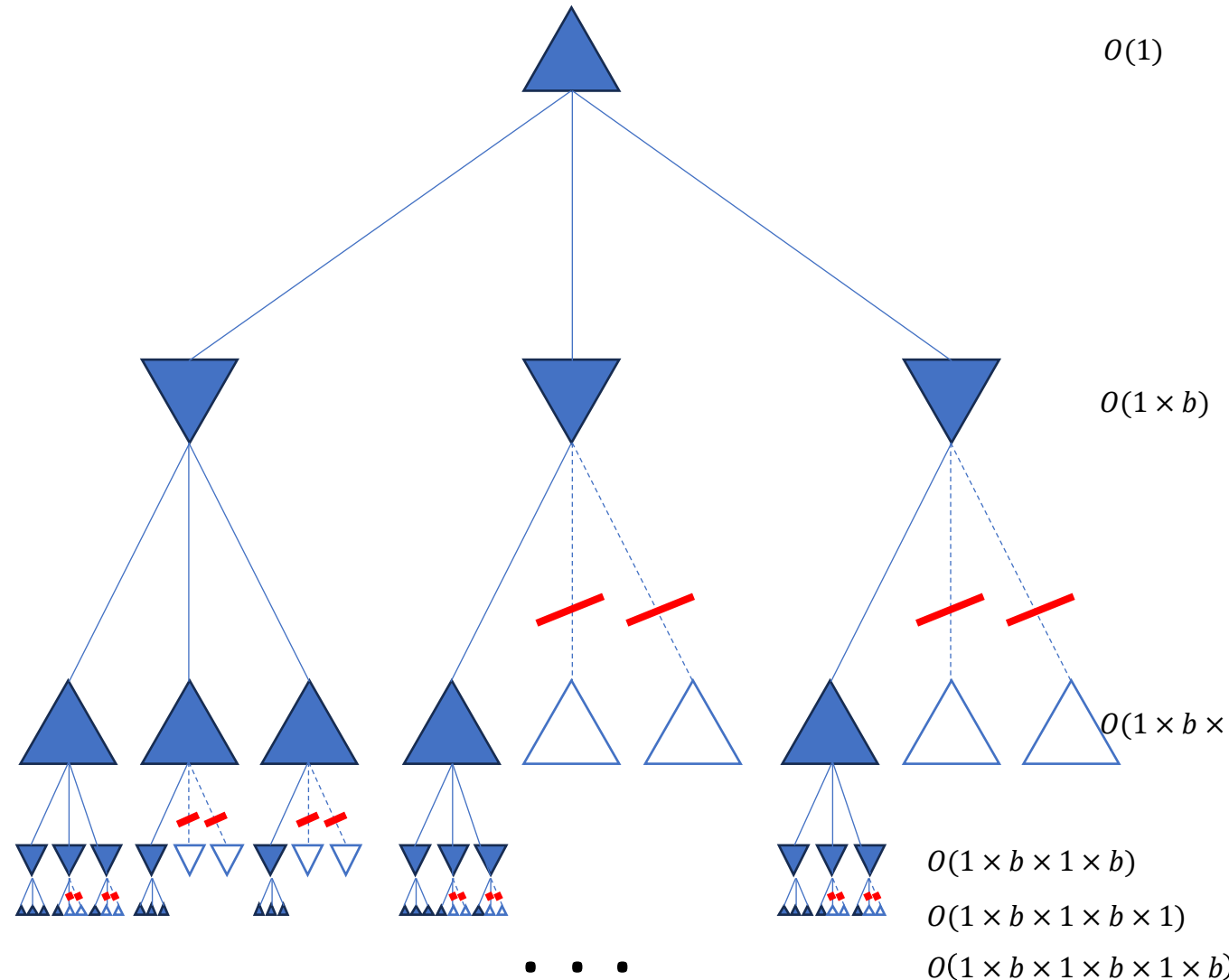
Alpha-Beta Pruning – Analysis

Theorem: Alpha-beta pruning does not alter the minimax **value** and the **optimal move** computed for the root node; it only reduces the number of nodes evaluated.

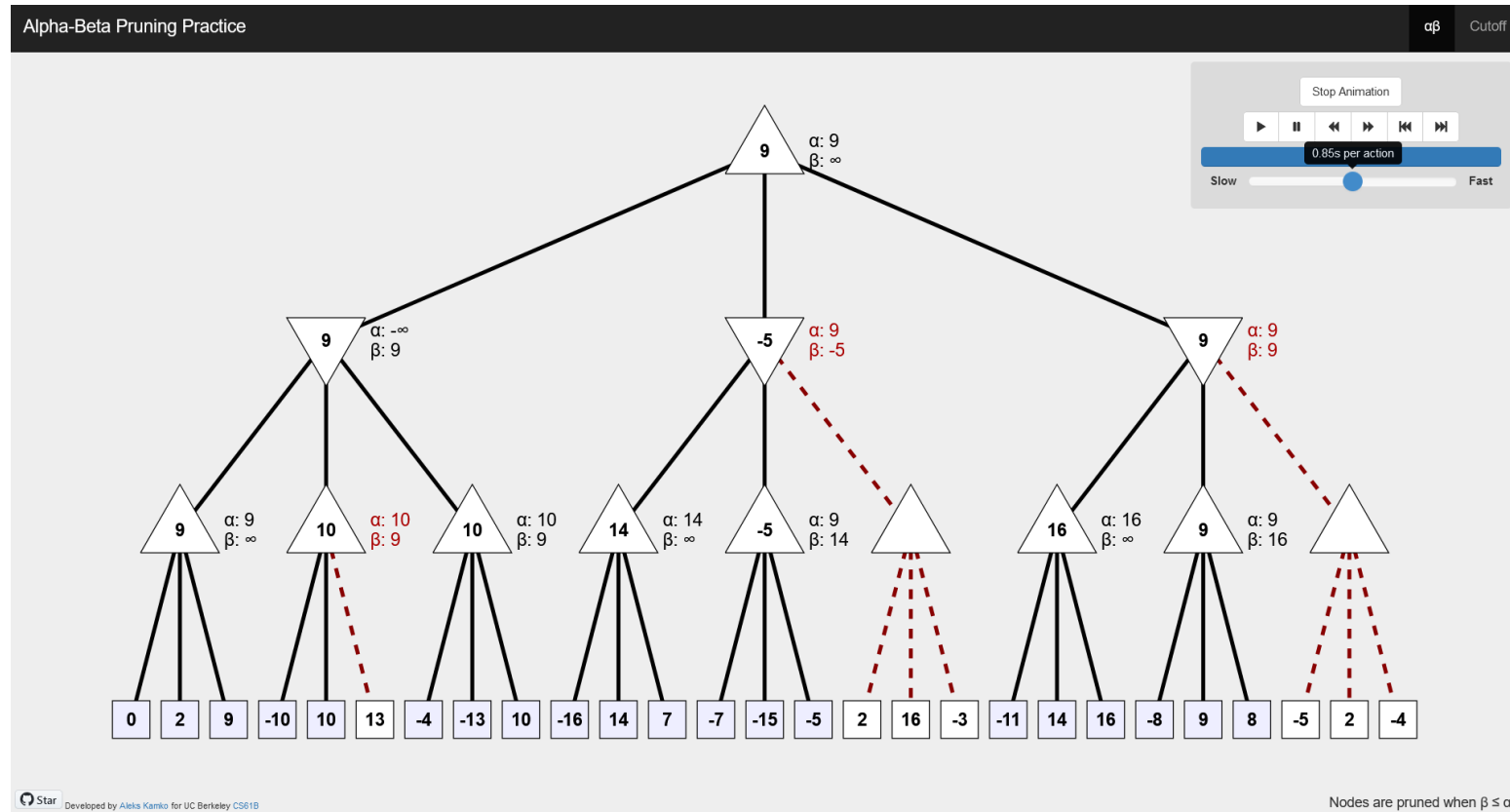
Logical view: $\max(10, \min(20, -30, ?)) = 10$

Making full use of alpha-beta pruning requires **meta-reasoning**: reasoning about which computations to do first.

Theorem: Good move ordering increases alpha-beta pruning effectiveness; with “perfect ordering”, its complexity is $O\left(b^{\frac{m}{2}}\right)$



Resources



<https://pascscha.ch/info2/abTreePractice/>

Note: can't be accessed from NUS network

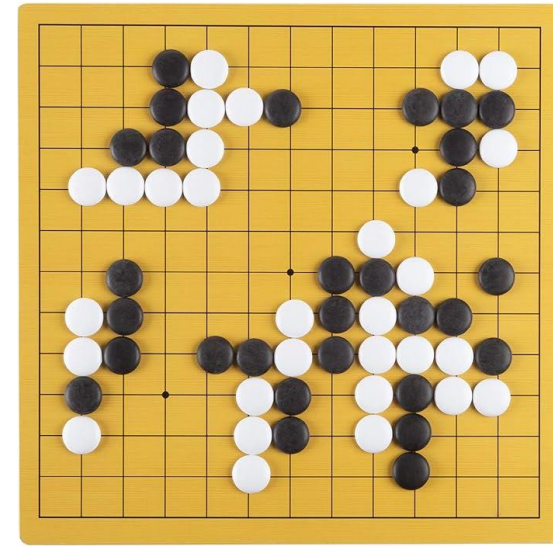
“Real-world” Games



Credit: chess.com

$b \approx 35, m \approx 80$ for “reasonable” games

Time complexity: $b^m = 35^{80} \approx 10^{123}$



Credit: Amazon.sg

$b \approx 250, m \approx 150$ for “reasonable” games

Time complexity: $b^m = 250^{150} \approx 10^{360}$

Number of particles in the universe: $\sim 10^{80}$

Imperfect Decisions by Using Cutoff

In real-world games, calculating the utility of a state by fully exploring the entire game tree with Minimax is often computationally infeasible (even with alpha-beta pruning).

Instead of evaluating the full tree,

- We can apply a **cutoff** strategy, halting the search in the middle, and
- Estimate the value of (mid-game) states using an **evaluation function**.

Use function **is_cutoff**(state):

- Return True if state is terminal or cutoff condition is satisfied.
- The cutoff condition is typically based on constraints such as the available computational budget (e.g., **time** or **search depth**).

Evaluation Function

- **Evaluation function** is a function that assigns a score to a game state, indicating how favorable it is for a player.
 - If a terminal state: use $utility(state)$
 - Otherwise: use a **heuristic**
- **Heuristic function** is a function that *estimates* the utility of a state.
 - A heuristic should fall somewhere between the minimum and maximum utility, such as between a loss and a win.
 - i.e., $\min_s utility(s) \leq heuristic(state) \leq \max_s utility(s)$, for all states
 - Example: $utility(loss) \leq heuristic(state) \leq utility(win)$, for all states
 - Computation must not be too expensive
 - In adversarial search, admissibility and consistency properties do not apply

“Inventing” a Heuristic

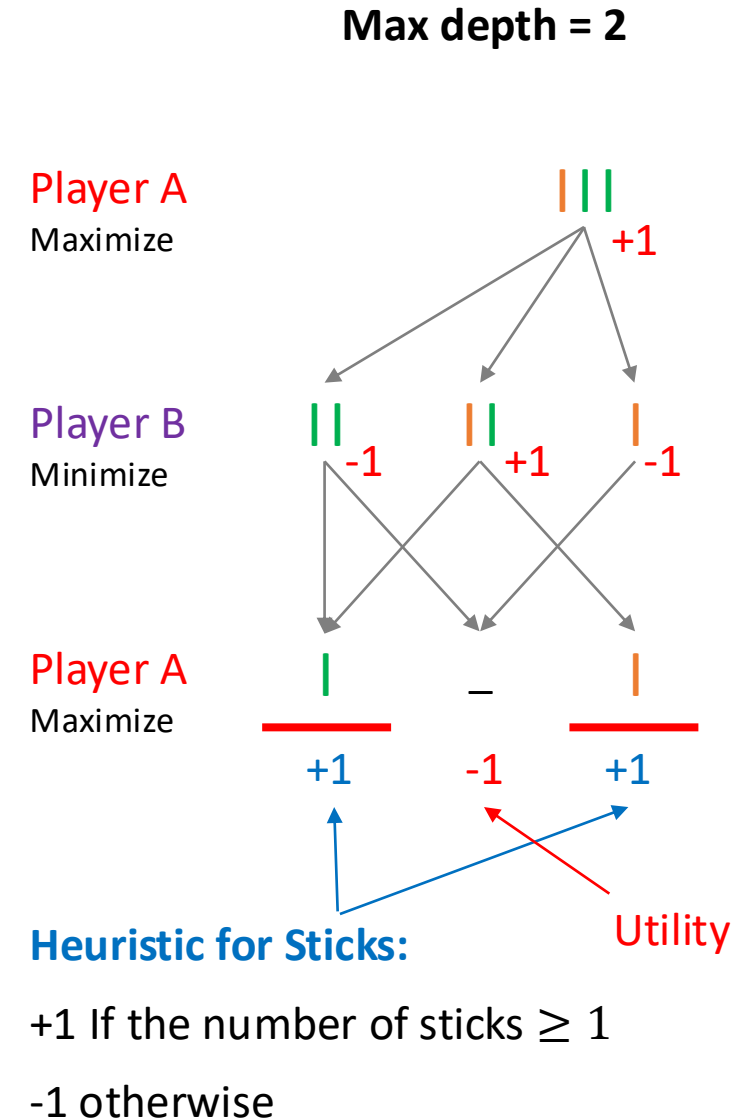
- Relaxed game – in principle, but usually too costly in practice
 - **Relaxed Chess**: each piece can teleport and perform self-sacrificial attack
 - Heuristic: number of agent’s pieces – number of opponent’s pieces
- Expert knowledge
 - **Chess**: often uses features like material count, mobility, safety, etc.
- Learn from data
 - Heuristic function could be learned from labeled (state, utility) dataset. The dataset could be generated using some expert agent or by solving the game (or some portions of the game)

Minimax with Cutoff

```
def minimax_with_cutoff(state):
    (action, v) = max_value(state)
    return action

def max_value(state):
    if is_cutoff(state): return eval(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state))
    return v

def min_value(state):
    if is_cutoff(state): return eval(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state))
    return v
```



Minimax with Cutoff – Optimality

Theorem: When minimax uses a cutoff and relies on an evaluation function to estimate the utility, it returns the best move according to that estimate.

Since the evaluation function is only a guess of how good a state really is, this move might not be the truly best move (i.e., optimal) — it's only as good as the estimate it's based on.

Real-world uses

- IBM Deep Blue: first to beat best human player in chess
 - Minimax alpha-beta pruning + expert evaluation function.
- Stockfish: one of the strongest chess AI today
 - Minimax alpha-beta pruning + neural networks evaluation function (NNUE).
- Many turn based strategy games use minimax alpha-beta pruning.

Summary

- Local search
 - Problems with large state space, good enough solution is okay
 - Hill climbing: pick the best the neighbour, repeat
 - State space: local/global maxima, shoulder
- Adversarial search
 - Competitive two-player turn-taking zero-sum games
 - Minimax: max of min value
 - Alpha-beta pruning: prune branches that don't affect decision
 - Handling large game trees: cutoff, evaluation/heuristic function

Further Reading (Optional)

- Local Search
 - Simulated annealing (AIAMA 4.1.2)
 - Escape techniques
 - Random restarts (AIAMA 4.1.1)
 - Tabu search (AIAMA Ch. 4, pg. 154)
 - Local beam search (AIAMA 4.1.3)
 - Genetic algorithm (AIAMA 4.1.4)
- Adversarial Search
 - Move ordering in alpha-beta pruning (AIAMA 5.2.4)
 - Optimizing the search with lookup (AIAMA 5.3.4)
 - Monte-Carlo tree search (AIAMA 5.4)
 - AlphaGo (Mastering the game of Go without human knowledge, Silver et al, 2017)

Coming Up Next Week

- **Intro to Machine Learning**
- **Decision Trees**
 - Decision Tree Learning
 - Entropy and Information Gain
 - Different types of attributes
 - Pruning

To Do

- **Lecture Training 3**
 - +550 EXP
 - +100 Early bird bonus
- **Tutorial already starts this week!**

Appendix

Alpha-beta Pruning

α = highest value for MAX
 β = lowest value for MAX

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

Alpha-beta Pruning

α = highest value for MAX
 β = lowest value for MAX

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

MAX

MIN



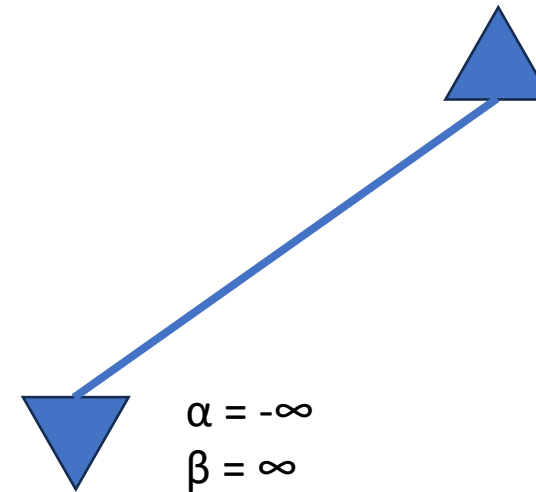
$\alpha = -\infty$
 $\beta = \infty$

Alpha-beta Pruning

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

MAX

MIN



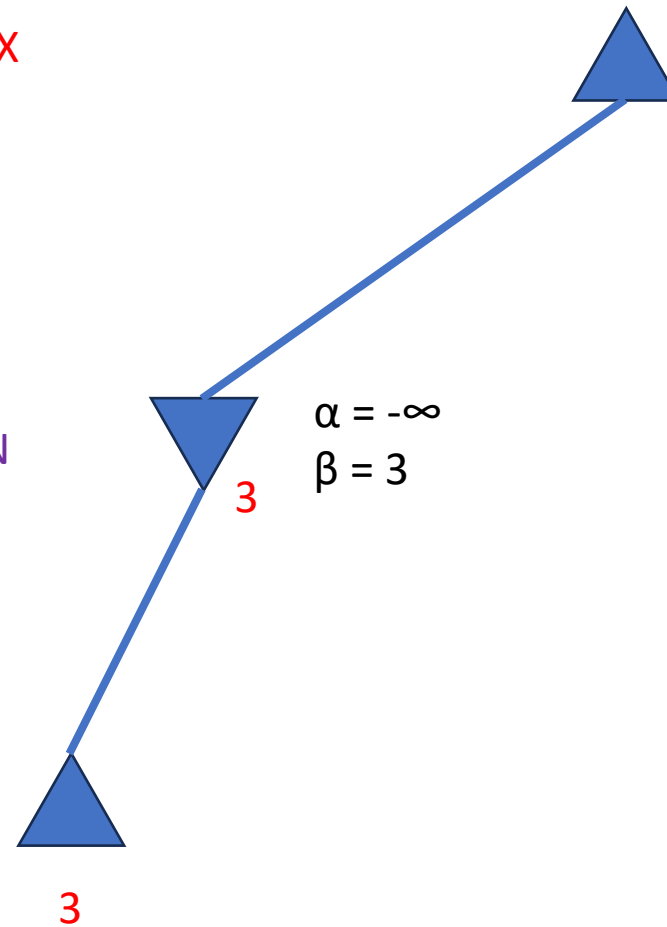
α = highest value for MAX
 β = lowest value for MAX

Alpha-beta Pruning

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

MAX

MIN



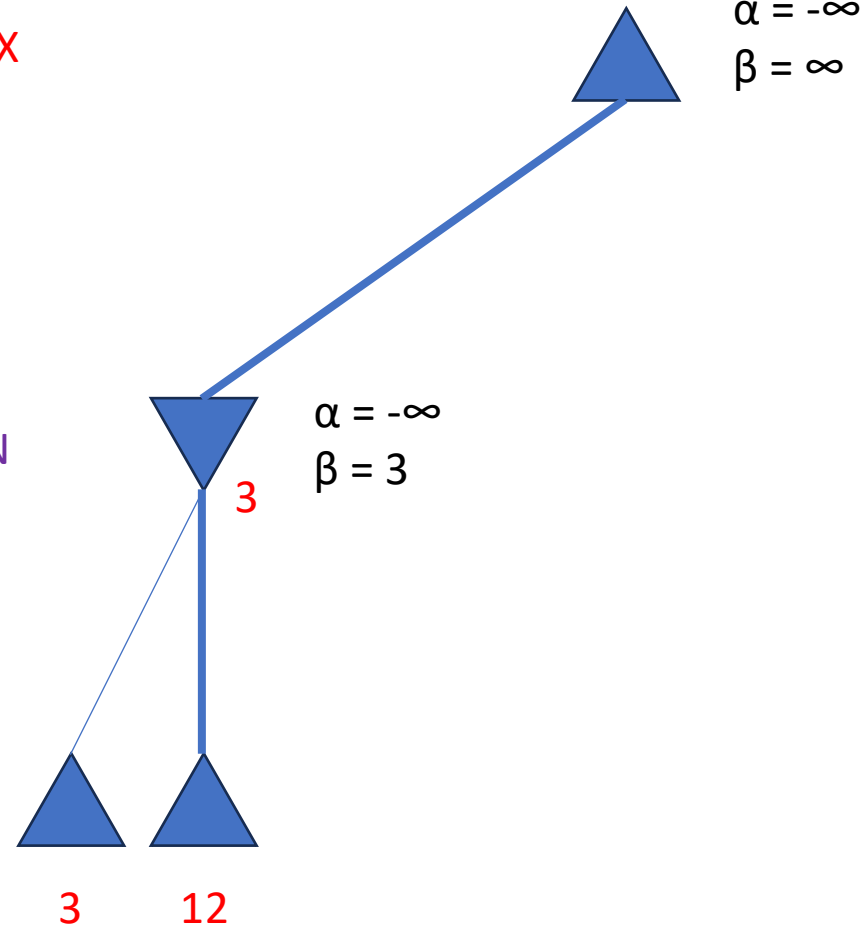
α = highest value for MAX
 β = lowest value for MAX

Alpha-beta Pruning

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

MAX

MIN



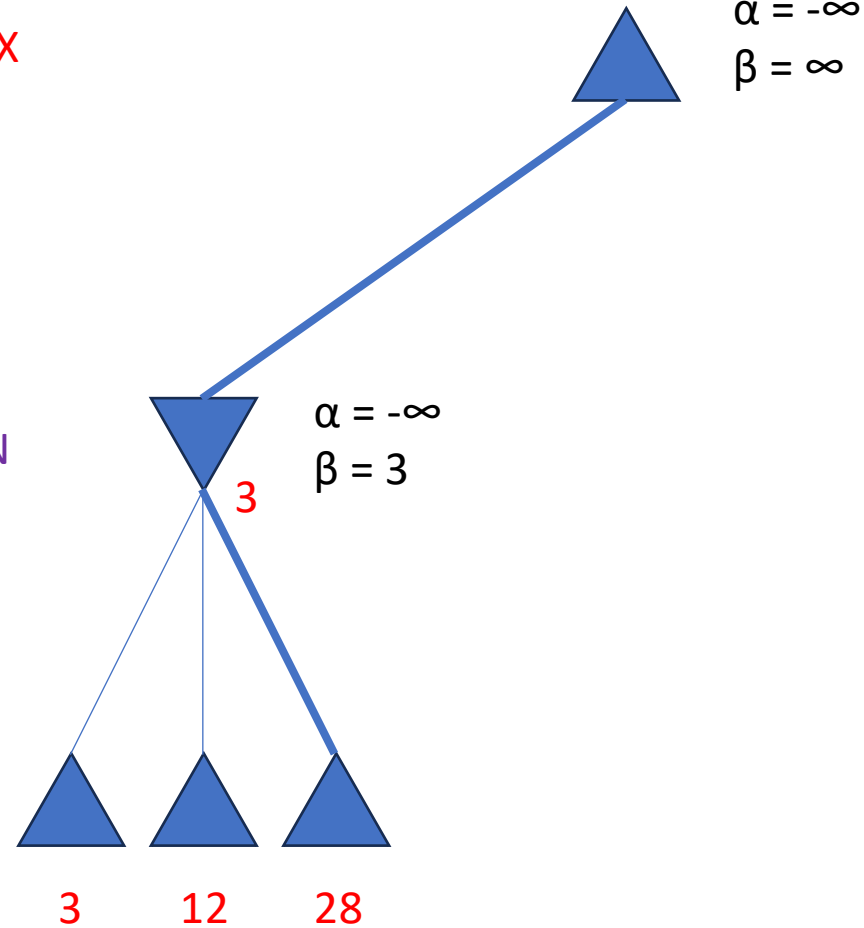
α = highest value for MAX
 β = lowest value for MAX

Alpha-beta Pruning

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

MAX

MIN

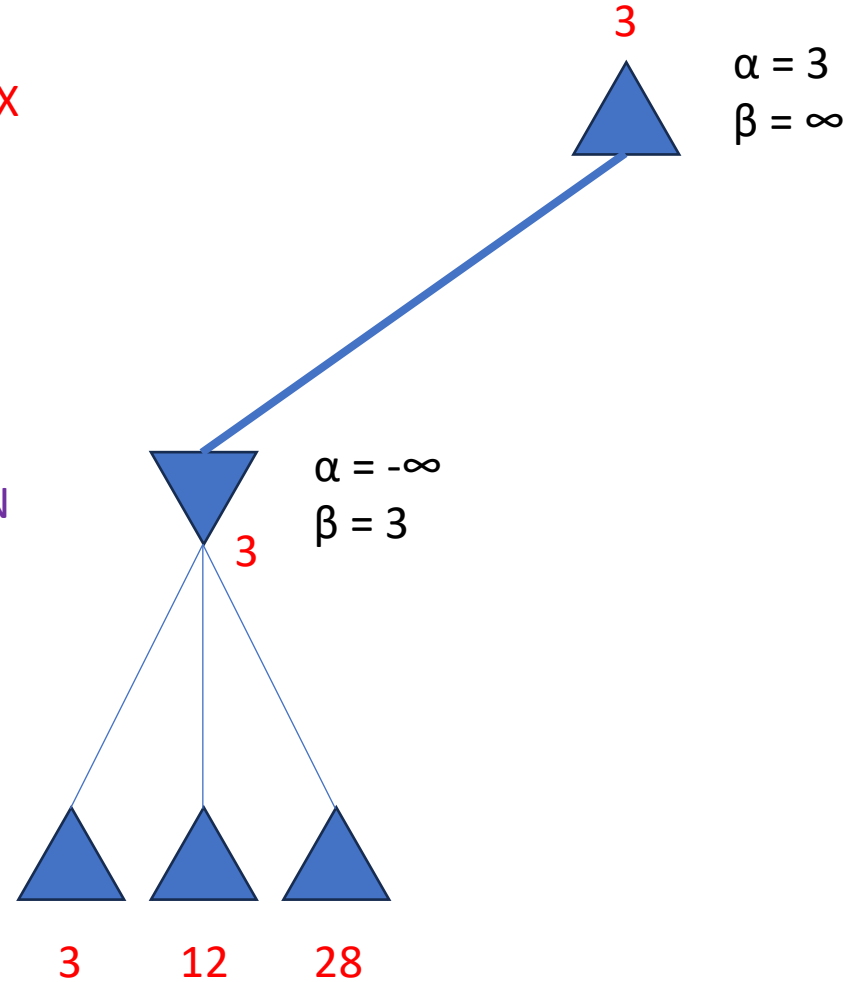


Alpha-beta Pruning

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

MAX

MIN

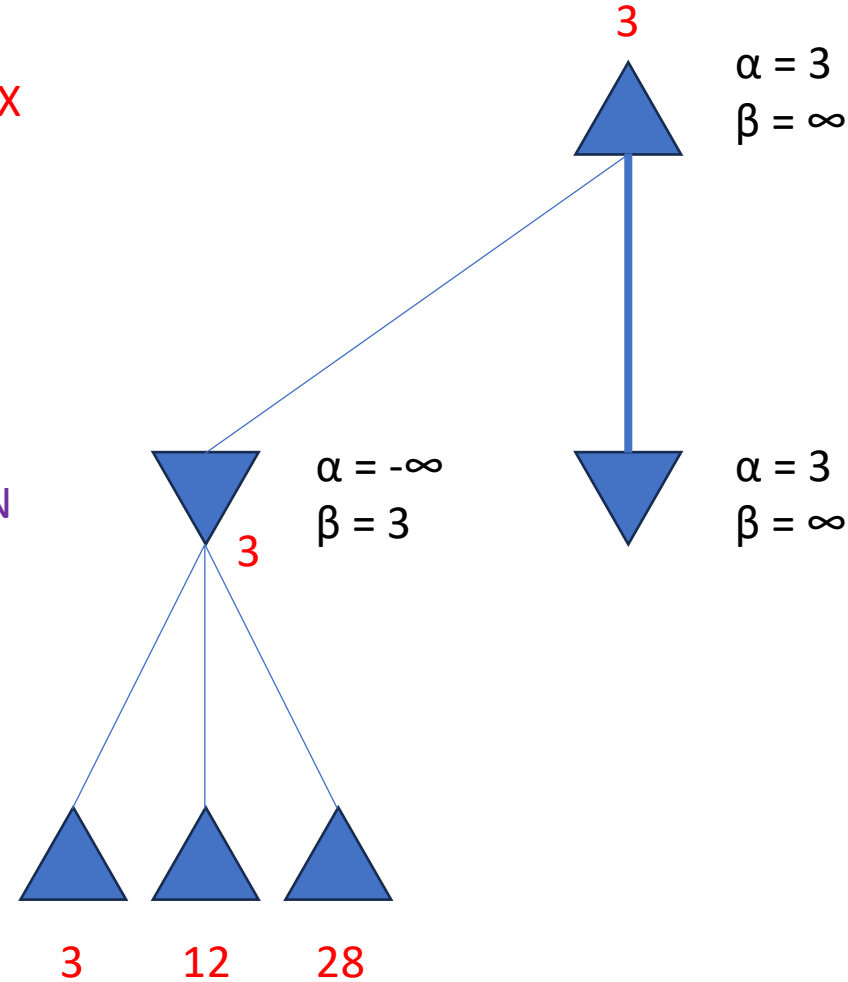


Alpha-beta Pruning

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

MAX

MIN



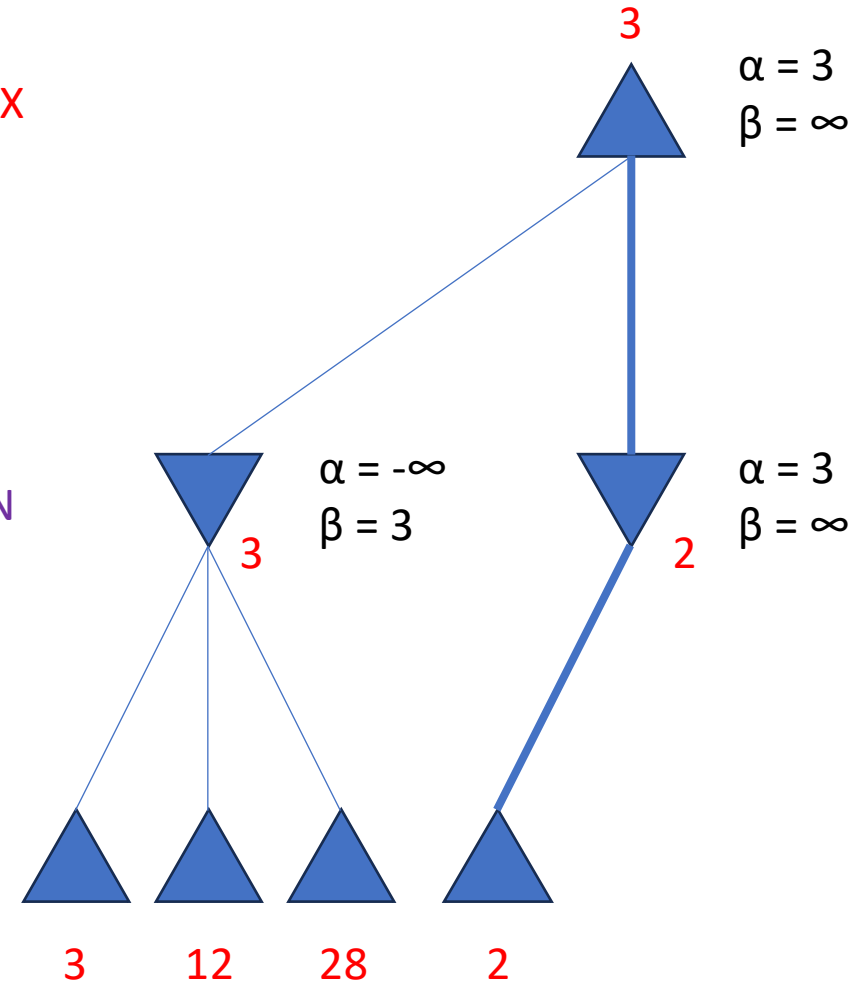
α = highest value for MAX
 β = lowest value for MAX

Alpha-beta Pruning

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

MAX

MIN



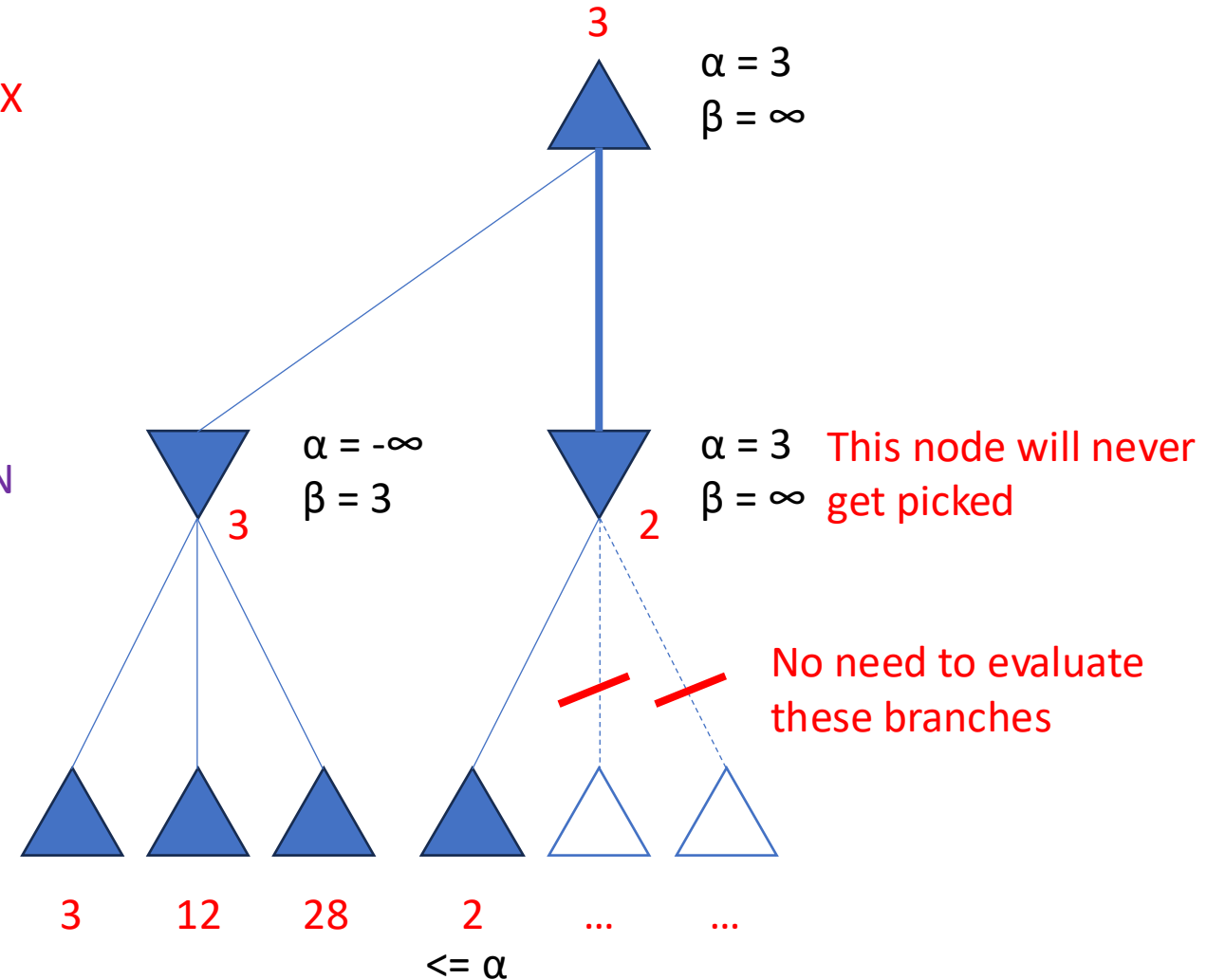
α = highest value for MAX
 β = lowest value for MAX

Alpha-beta Pruning

```
def alpha_beta_search(state):  
    (action, v) = max_value(state, -∞, ∞)  
    return action  
  
def max_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = -∞  
    for next_state in expand(state):  
        v = max(v, min_value(next_state, α, β))  
        α = max(α, v)  
        if v >= β: return v  
    return v  
  
def min_value(state, α, β):  
    if is_terminal(state): return utility(state)  
    v = ∞  
    for next_state in expand(state):  
        v = min(v, max_value(next_state, α, β))  
        β = min(β, v)  
        if v <= α: return v  
    return v
```

MAX

MIN

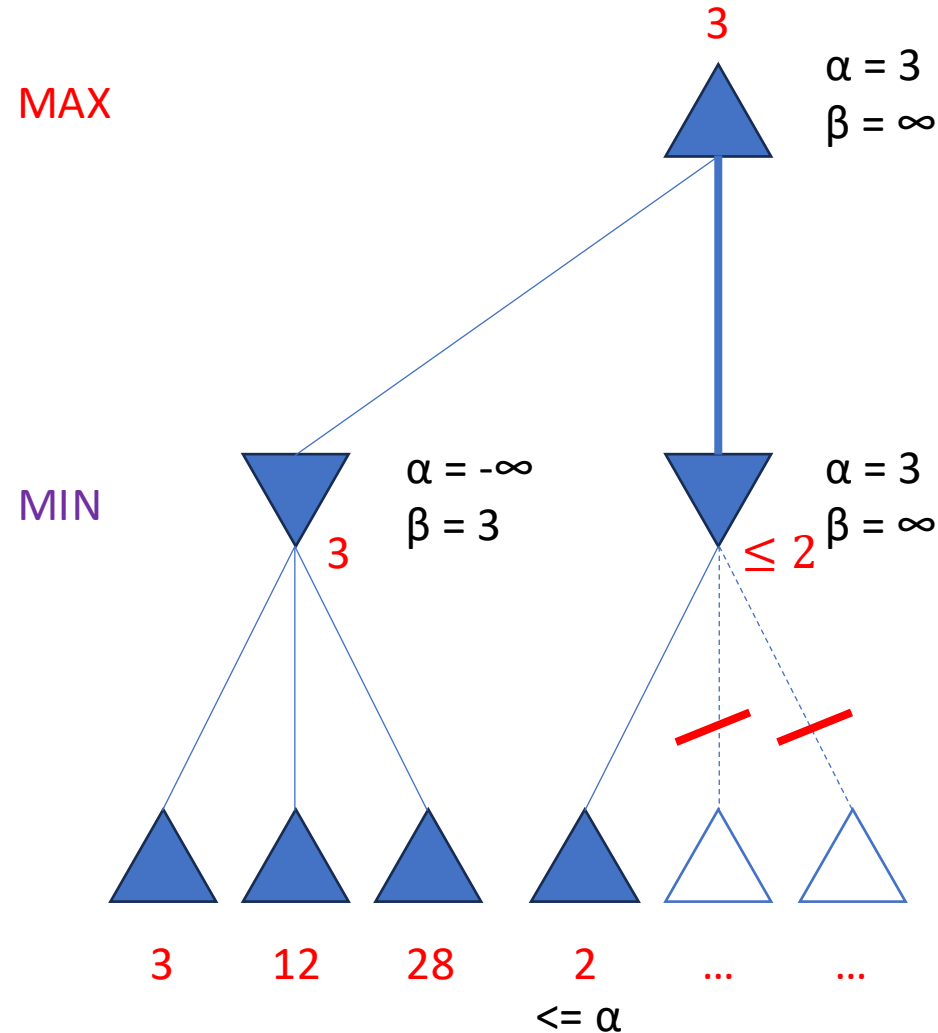


Alpha-beta Pruning

```
def alpha_beta_search(state):
    (action, v) = max_value(state,  $-\infty$ ,  $\infty$ )
    return action

def max_value(state,  $\alpha$ ,  $\beta$ ):
    if is_terminal(state): return utility(state)
    v =  $-\infty$ 
    for next_state in expand(state):
        v = max(v, min_value(next_state,  $\alpha$ ,  $\beta$ ))
         $\alpha = \max(\alpha, v)$ 
        if v >=  $\beta$ : return v
    return v

def min_value(state,  $\alpha$ ,  $\beta$ ):
    if is_terminal(state): return utility(state)
    v =  $\infty$ 
    for next_state in expand(state):
        v = min(v, max_value(next_state,  $\alpha$ ,  $\beta$ ))
         $\beta = \min(\beta, v)$ 
        if v <=  $\alpha$ : return v
    return v
```



Alpha-beta Pruning

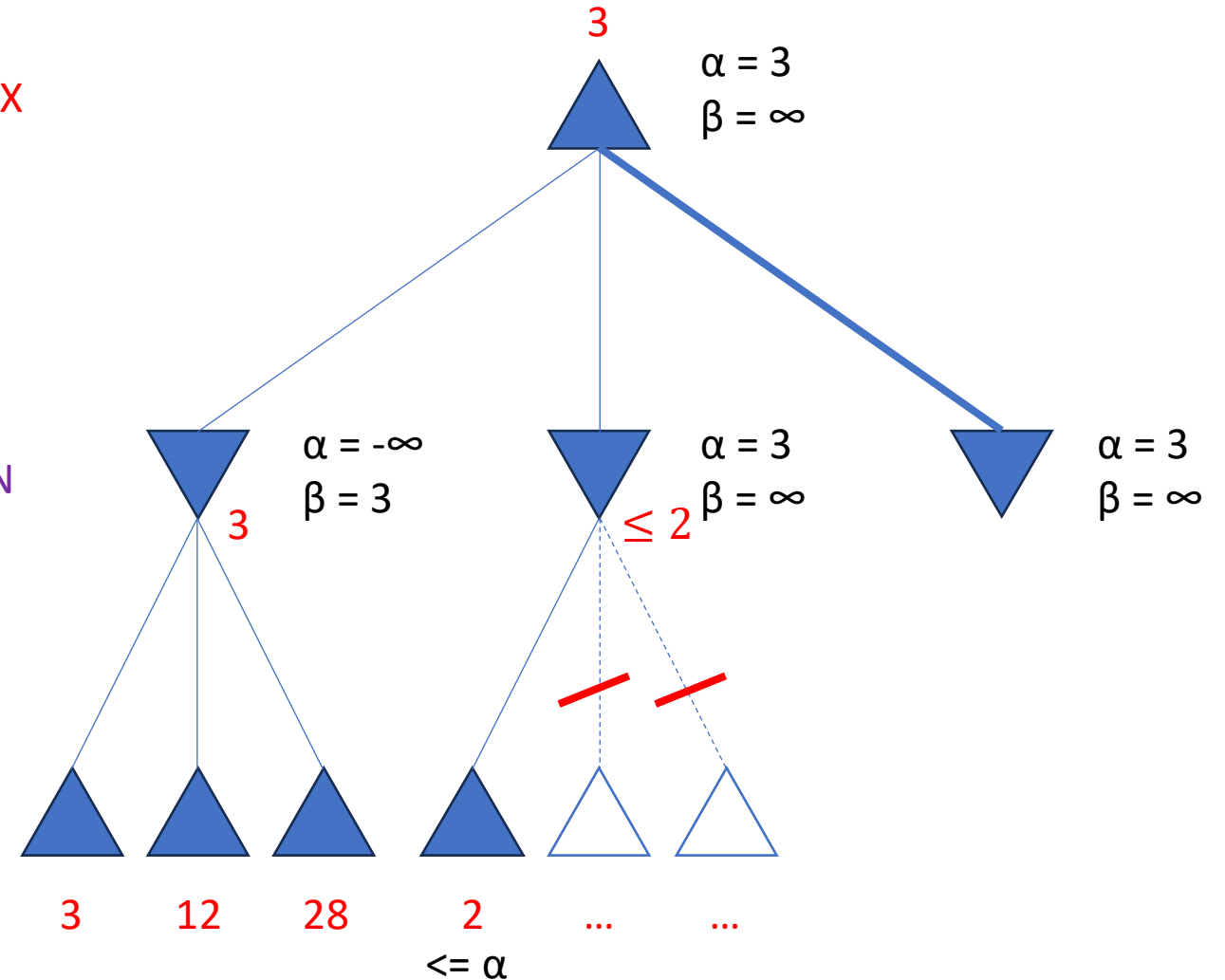
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

MAX

MIN



Alpha-beta Pruning

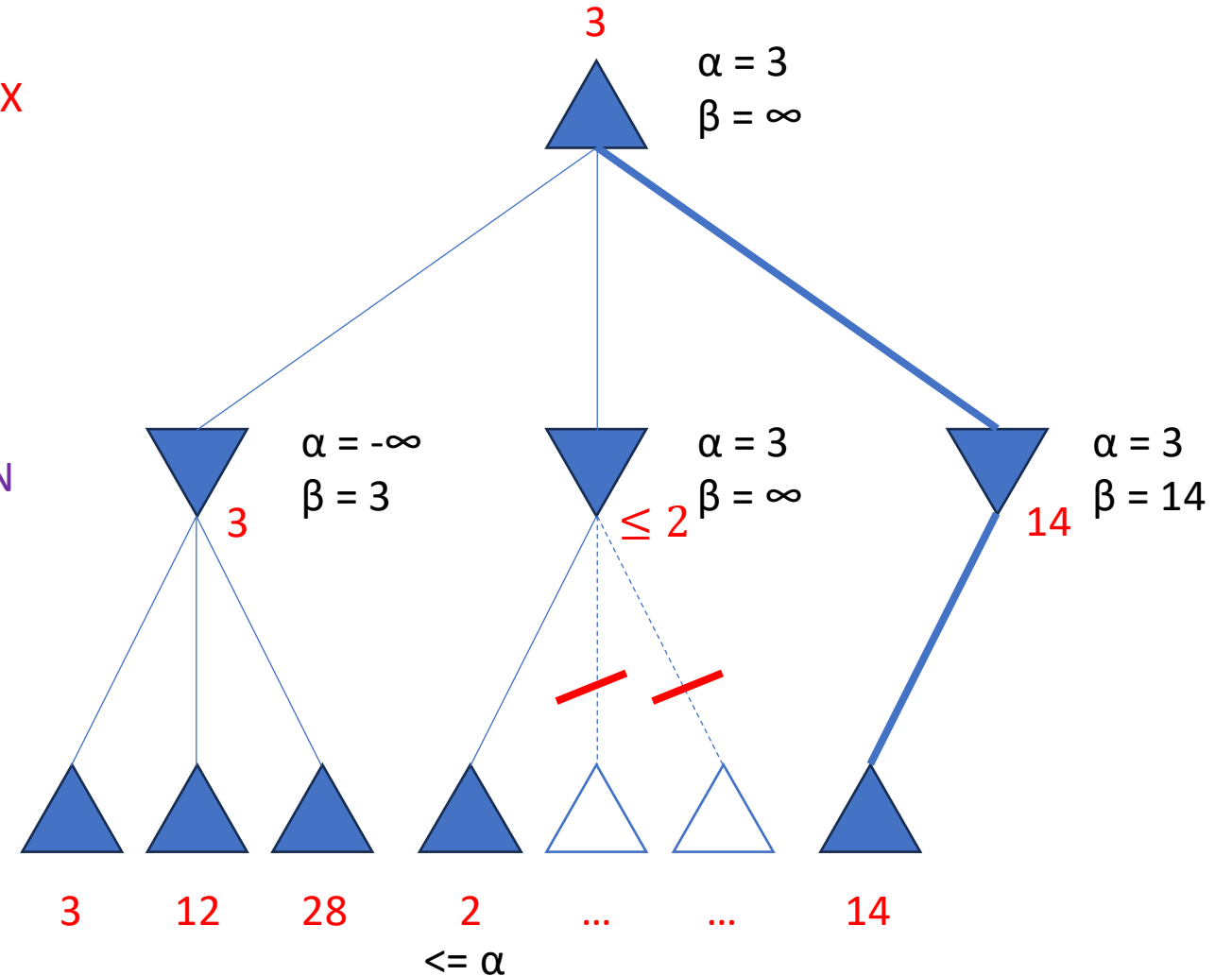
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

MAX

MIN



Alpha-beta Pruning

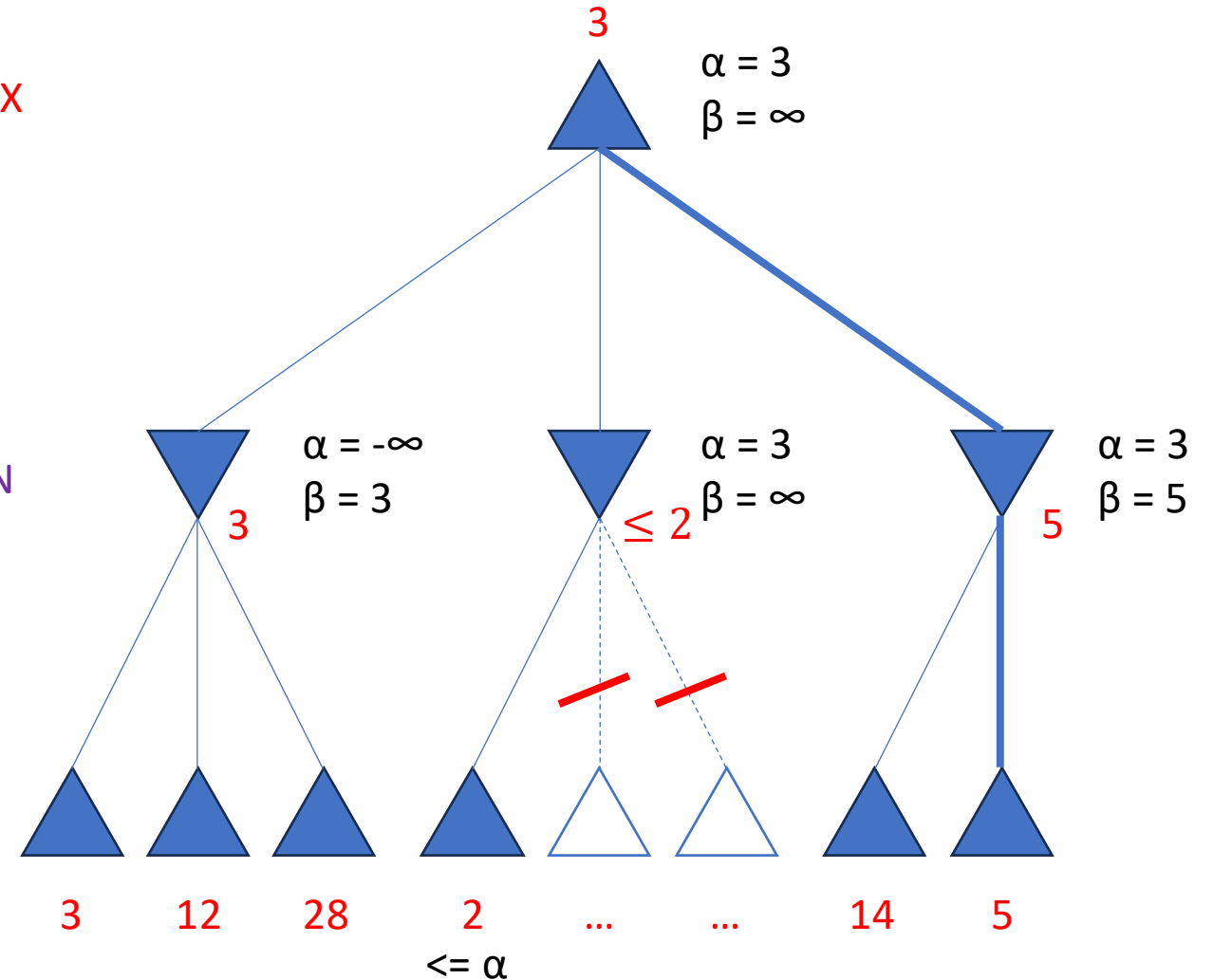
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

MAX

MIN



Alpha-beta Pruning

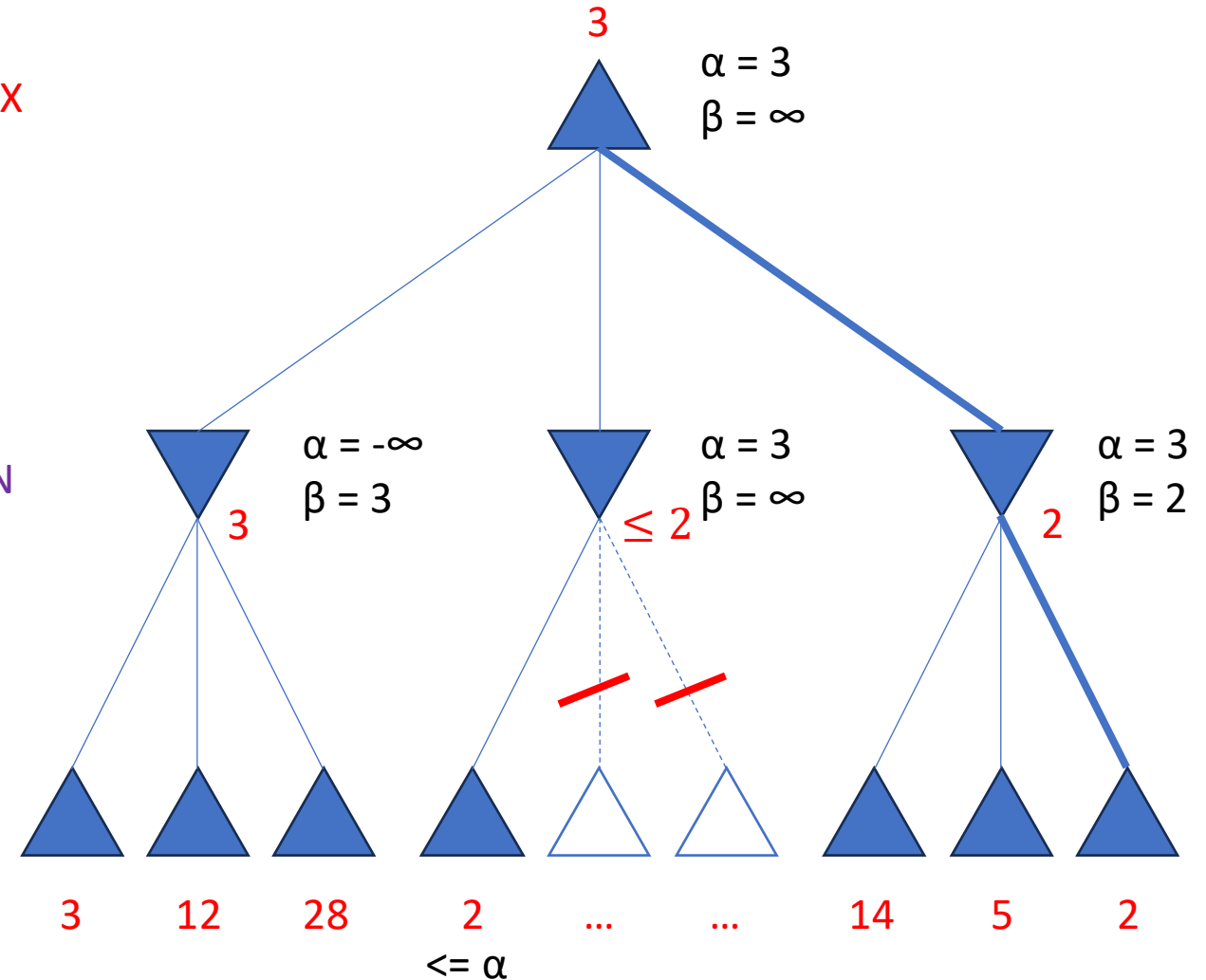
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

MAX

MIN



Alpha-beta Pruning

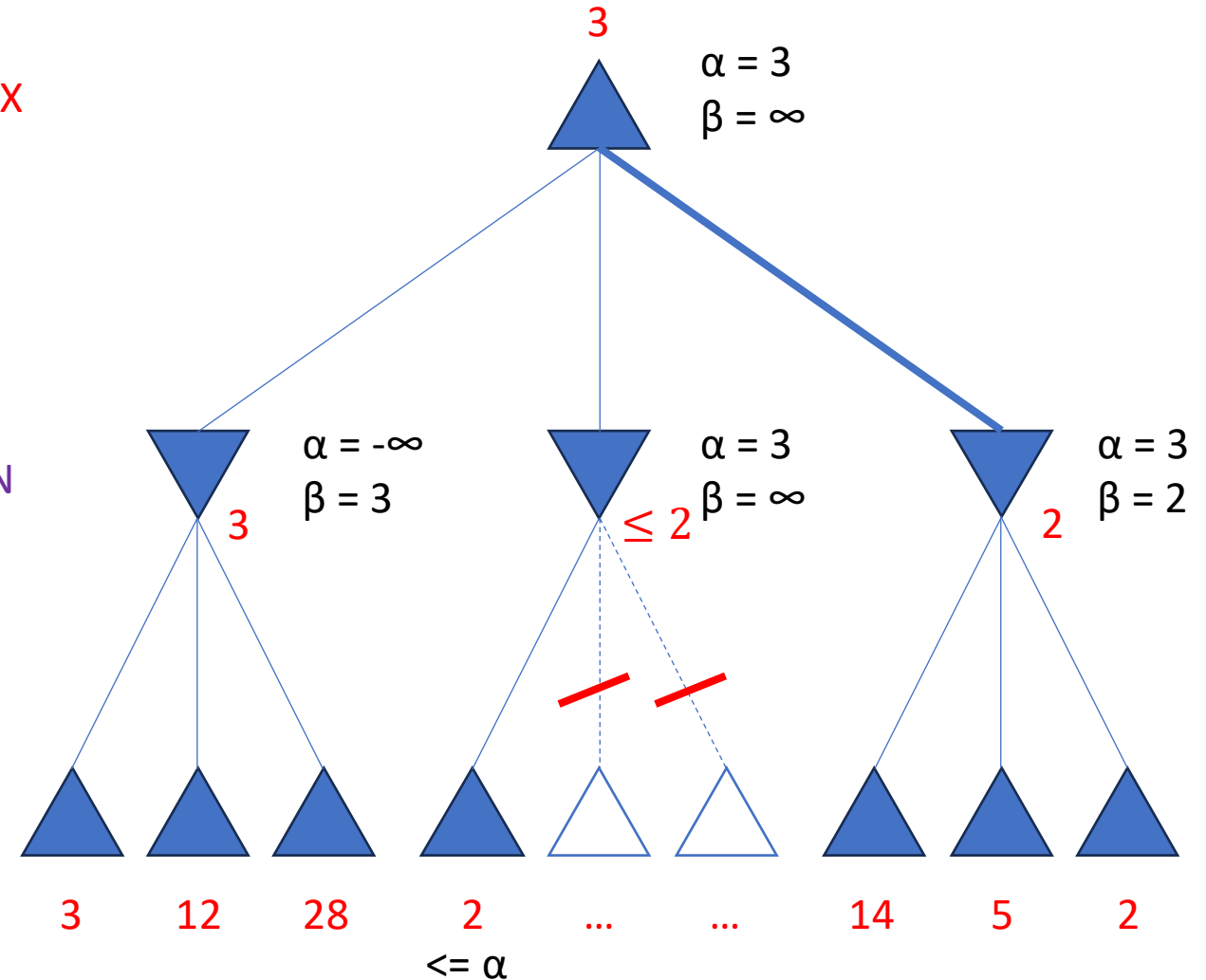
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

MAX

MIN



Alpha-beta Pruning

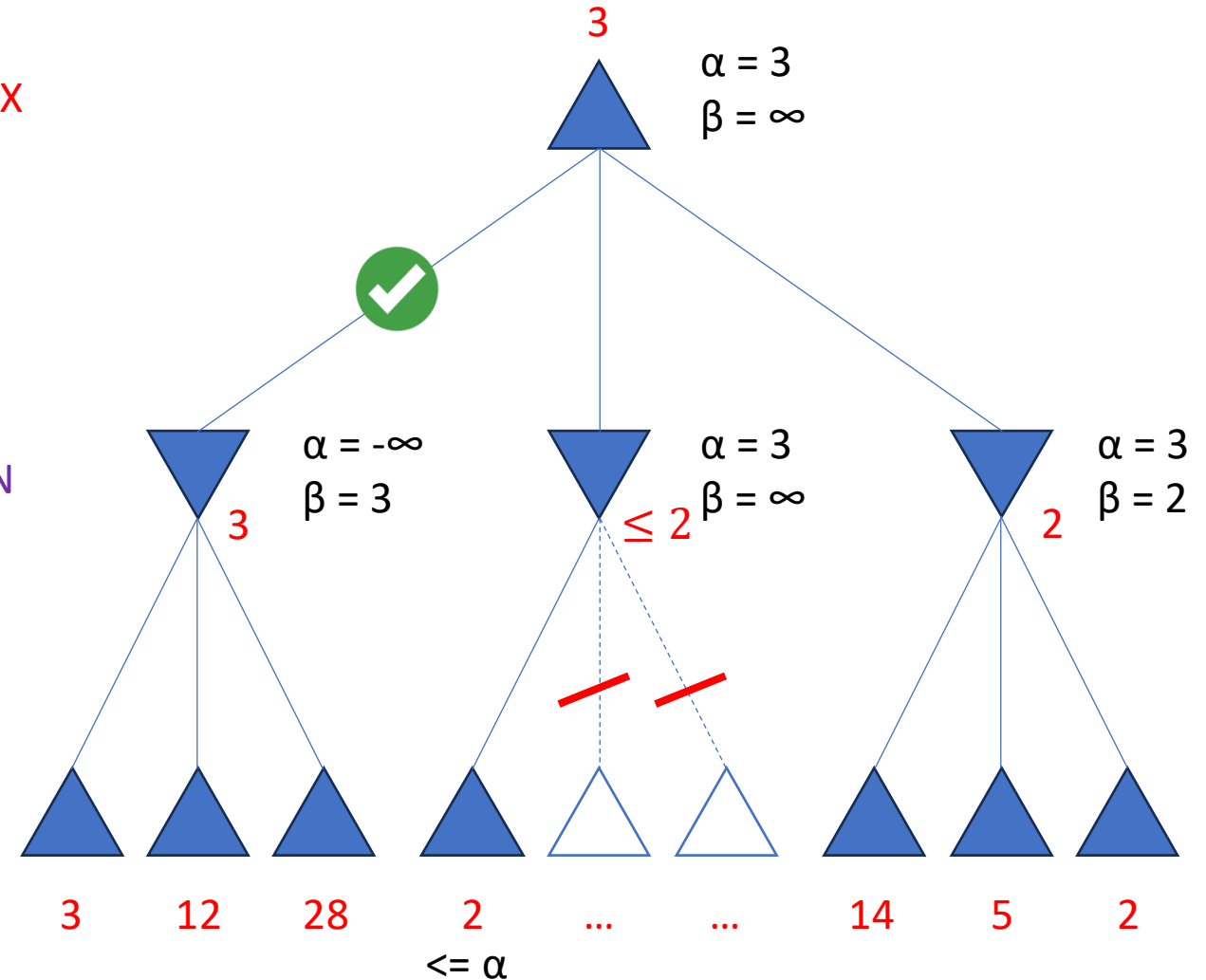
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

MAX

MIN



Summary: Local Search

- There are problems with a very large state space
 - Good enough solution is okay
- Local Search
 - Hill-climbing (greedy): **pick the best** among neighbours, repeat
- State space landscape
 - Local maximum
 - Global maximum
 - Shoulder

Summary: Adversarial Search

- Games vs Search Problems:
 - Search problems: **predictable** world, deterministic and stochastic
 - Games: “**unpredictable**” opponent
- Minimax:
 - Assume **opponent behaves optimally**
 - Pick a move that **maximizes** player’s **utility**
- Alpha-beta pruning
 - **Prune branches** that are useless (**will not change the decision**)
- Handling large/infinite game trees:
 - **Cutoff** the search in the middle of the game and use **evaluation function**